Singapore University of Technology and Design

50.039 Theory and Practices of Deep Learning

Final Report

# Real-time Aircraft Engine Abnormal Remaining Useful Life (RUL) Prediction


**Group 5**

| **Name** | **Student ID** |
|---|---|
| Gan Jessica Loraine Ngo | 1006942 |
| Toh Hengyi Lucas | 1006061 |
| Palinya Sengdalavong | 1006926 |

*1. Introduction* ..... *2*
*2. State-of-The-Art* ..... *3*
*3. Implementation* ..... *3*
3.1. Data Exploration (EDA) ..... 4
3.1.1. Dataset Overview ..... 4
3.1.2. Sensor Visualisation ..... 5
3.1.3. Cycle Distribution ..... 6
3.1.4. Variance Analysis ..... 8
3.1.5. Correlation Analysis ..... 9
3.2. Data Preprocessing ..... 10
3.2.1. RUL Label Reconstruction ..... 10
3.2.2. Min-max Normalisation of Features ..... 11
3.3. Feature Engineering ..... 12
3.4. Data Splitting, Model Saving, and Loading ..... 13
3.5. Model Architectures ..... 14
3.6. Reproducibility ..... 22
*4. Malfunction* ..... *24*
*5. Evaluation* ..... *25*
5.1. RNN-Based Training Results and Selection of Final Comparison Dataset ..... 26
5.2. Transformer Training Results ..... 32
5.3. Hybrid-Transformer Training Results ..... 34
5.4. Best Results for Each Model (Before Finetuning) ..... 36
5.5. Best Results for Each Model (After Finetuning) ..... 36
5.6. Benchmarking with SOTA ..... 37
*6. Intepretation of Results* ..... *37*
*7. Anomaly Detection from Regression to Classification* ..... *38*
*8. Conclusion* ..... *41*
*9. Contribution* ..... *41*
*10. References* ..... *42*

# 1. Introduction

The aviation industry faces significant challenges regarding safety and operational efficiency. Predictive maintenance (PdM) is essential to identify “abnormal” engine behaviour before catastrophic failure occurs, reducing costs from unscheduled repairs and fatal accidents.

While traditional maintenance relied on scheduled checks. PdM utilizes historical and real-time sensory data to model degradation patterns. By analysing temporal dependencies, models can predict the state of a component based on its operational history.

A key challenge in this domain is the inherently imbalanced nature of engine health data (i.e. most operational cycles represent normal, healthy engine states, while only a small fraction captures the critical degradation window preceding failure). This project therefore frames the problem as anomaly detection: using regression-based RUL prediction as an intermediate representation to ultimately classify whether an engine is entering a dangerous operational state, where anomalous (near-failure) samples are significantly outnumbered by normal ones.

# 2. State-of-The-Art

A key direction in the literature is the convolution-Transformer family. For example, the GCU-Transformer integrates a gated convolutional unit before the Transformer encoder so that local temporal patterns are extracted before self-attention models global context (Y. Mo, Q. Wu, X. Li, and B. Huang, 2021). This is useful especially in sensor data because degradation often appears first as short-term changes in sensor trends rather than as a clear global pattern. In the same line, DSFormer introduces dual-scale temporal modeling and convolutional components to capture both short-term and longer-horizon information, which helps when the degradation trajectory changes across multiple time scales (J. Li et al., 2024). Another strong variant is AGCTE, which combines an adaptive graph convolutional network with a Transformer encoder to better model inter-sensor relationships (J. Li et al., 2024). More broadly, hybrid designs that place convolution before Transformer stages are motivated by the idea that convolution is better at extracting local patterns in early stages (Tan et al., 2021).

# 3. Implementation

## 3.1. Data Exploration (EDA)

### 3.1.1. Dataset Overview

The analysis uses the NASA CMAPSS (Commercial Modular Aero-Propulsion System Simulation) dataset, specifically one main sub-dataset: FD001. FD001 contains time-series sensor readings from turbofan engines, structured with 26 columns: an engine ID (id), an operational cycle counter (cycle), three operational setting variables (setting1, setting2, setting3), and 21 sensor readings (s1 through s21). Additionally, accompanied with the data would be the Remaining Useful Life target feature that is constructed from taking the maximum cycles minus current cycles for the training sets, whereas for it is provided directly as a file for each engine in the test set.

For this project, we will be focusing on FD001 as our main dataset to train and finetune our models.

| **Dataset** | **Train Rows** | **Test Rows** | **Mean Cycle** | **Max Cycle** |
|---|---|---|---|---|
| FD001 | 20,631 | 13,096 | 108.8 | 362 |

*Figure 3.1.1.1: Size of Datasets*

| Subset | Operation Conditions | Fault modes | Use case |
|---|---|---|---|
| FD001 | 1 | 1 | Baseline single condition case |

*Figure 3.1.1.2: Characteristics of FD001*

| | FD001 | | |
|---|---|---|---|
| | min | mean | max |
| id | 1.0000 | 51.5066 | 100.0000 |
| cycle | 1.0000 | 108.8079 | 362.0000 |
| setting1 | -0.0087 | -0.0000 | 0.0087 |
| setting2 | -0.0006 | 0.0000 | 0.0006 |
| setting3 | 100.0000 | 100.0000 | 100.0000 |
| s1 | 518.6700 | 518.6700 | 518.6700 |
| s2 | 641.2100 | 642.6809 | 644.5300 |
| s3 | 1571.0400 | 1590.5231 | 1616.9100 |
| s4 | 1382.2500 | 1408.9338 | 1441.4900 |
| s5 | 14.6200 | 14.6200 | 14.6200 |
| s6 | 21.6000 | 21.6098 | 21.6100 |
| s7 | 549.8500 | 553.3677 | 556.0600 |
| s8 | 2387.9000 | 2388.0967 | 2388.5600 |
| s9 | 9021.7300 | 9065.2429 | 9244.5900 |
| s10 | 1.3000 | 1.3000 | 1.3000 |
| s11 | 46.8500 | 47.5412 | 48.5300 |
| s12 | 518.6900 | 521.4135 | 523.3800 |
| s13 | 2387.8800 | 2388.0962 | 2388.5600 |
| s14 | 8099.9400 | 8143.7527 | 8293.7200 |
| s15 | 8.3249 | 8.4421 | 8.5848 |
| s16 | 0.0300 | 0.0300 | 0.0300 |
| s17 | 388.0000 | 393.2107 | 400.0000 |
| s18 | 2388.0000 | 2388.0000 | 2388.0000 |
| s19 | 100.0000 | 100.0000 | 100.0000 |
| s20 | 38.1400 | 38.8163 | 39.4300 |
| s21 | 22.8942 | 23.2897 | 23.6184 |

*Figure 3.1.1.3: Characteristics of Features in FD001*

### 3.1.2. Sensor Visualisation

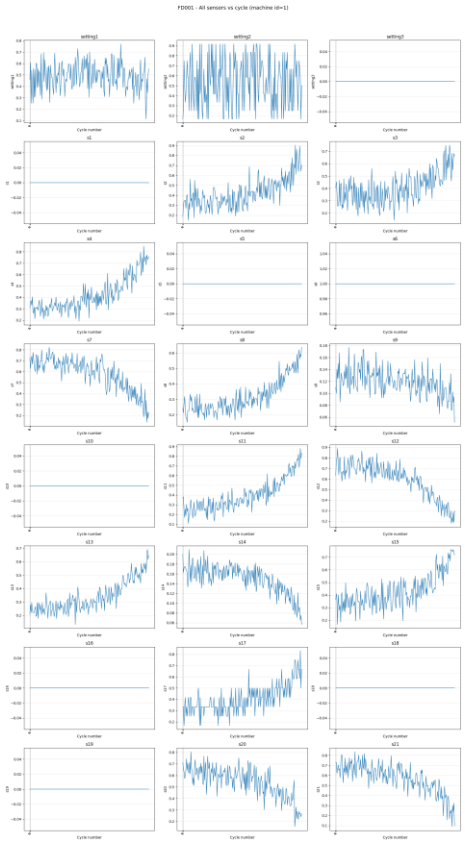


*Fig 3.1.2: Sensor Readings Across Cycles for ID 1 Engine for FD001*

To understand degradation patterns, all 21 sensor signals were plotted against operational cycles for individual engines. This per-engine visualisation revealed that different sensors exhibit varying degrees of monotonic trends as engines approach failure — some increasing (e.g., s12, s7), some decreasing (e.g., s11, s4), while others remain flat throughout the lifecycle. More importantly, this visual inspection tells us that for our main dataset FD001, there may exist some features that do not contribute significantly to engine degradation, which we may remove to remove noise in the dataset that do not contribute meaningfully. This is further studied in the Section 3.1.4 for a quantitative view.

### 3.1.3. Cycle Distribution

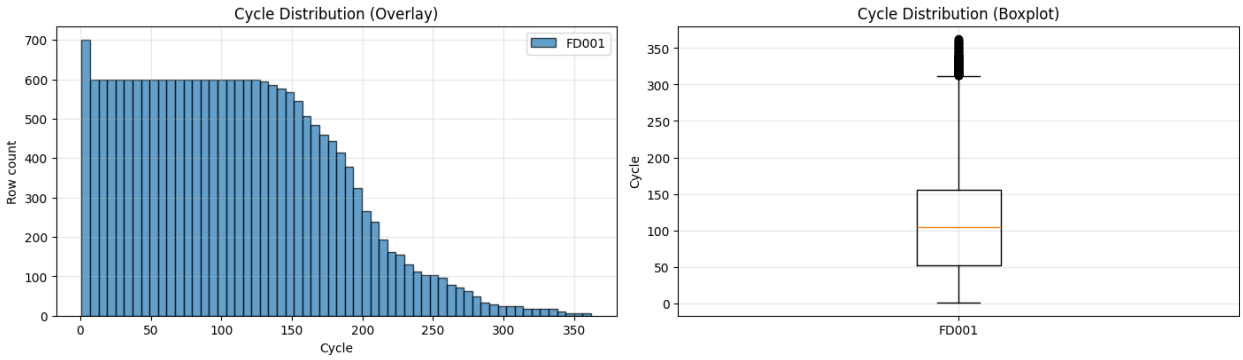



*Fig 3.1.3.1: Cycle Distribution for FD001 Dataset*

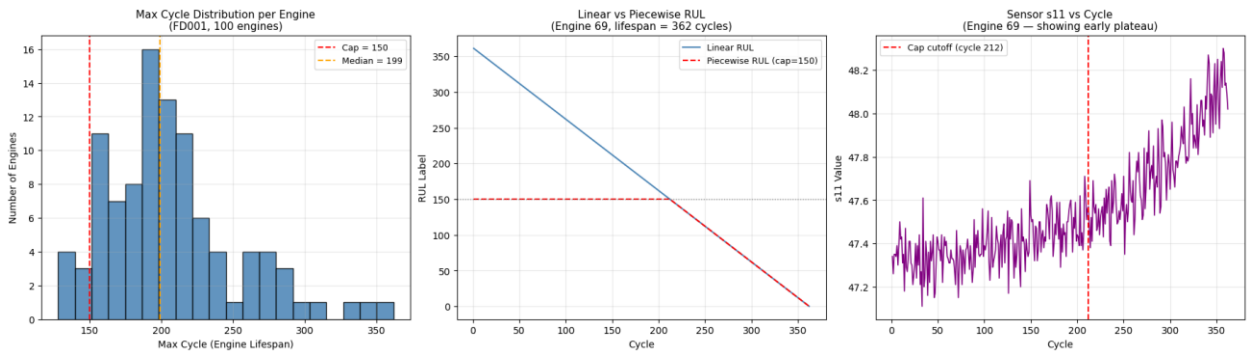



*Fig 3.1.3.2 Max Cycle Distribution and Engine 69 Example in FD001*

From the cycle distribution, we can determine the distribution of maximum engine cycles for all engines for both datasets. FD001 shows a right-skewed distributions, which indicates that a large proportion of engines fail relatively early as seen from the drop off at around ~125 cycles. This finding suggests to us that almost every engine has a long pre-degradation phase where most of the sensor readings are unlikely to contribute to any change of RUL.

For a clearer example, we can look at Engine ID 69 that has a lifespan of 362 cycles. We can see that the corresponding sensor reading is flat from cycle 1 to cycle 212 before beginning its characteristic decline. Under linear RUL, rows in that flat zone are labelled RUL = 362 down to 150, which represents a span of 212 unique labels attached to sensor readings that may be almost the same, which forces model to learn arbitrary distinctions from a noisy plateau.

This cycle distribution shows us a specific characteristic of the engine behaviour within FD001 dataset which is that most engines spend a significant proportion of life in a state that is no different than new/not degrading. This distribution also reveals the class imbalance inherent to the dataset: if failure is defined as RUL ≤ 30 cycles, only ~15% of training rows fall into the anomalous class, with the remaining ~85% representing normal operation.

### 3.1.4. Variance Analysis

| | FD001_train_variance |
|---|---|
| setting1 | 0.0158024 |
| setting2 | 0.0596426 |
| setting3 | 0 |
| s1 | 0 |
| s2 | 0.0226859 |
| s3 | 0.017866 |
| s4 | 0.0230841 |
| s5 | 0 |
| s6 | 0.0192928 |
| s7 | 0.0203139 |
| s8 | 0.0115678 |
| s9 | 0.00981855 |
| s10 | 0 |
| s11 | 0.0252748 |
| s12 | 0.024731 |
| s13 | 0.0111858 |
| s14 | 0.00969091 |
| s15 | 0.0208241 |
| s16 | 0 |
| s17 | 0.0166574 |
| s18 | 0 |
| s19 | 0 |
| s20 | 0.0196318 |
| s21 | 0.0223432 |

*Fig 3.1.4: Variance Analysis for FD001*

In addition to looking at the patterns of data, we also looked at the variance of each setting and feature. Based on the prior visual analysis of Engine ID 1, we notice that some of the features have no signal at all, which we presume is what creates the singular operational condition of FD001 dataset. This variance analysis hence informs us exactly which variables contribute towards changes in RUL, motivating our selection of features in Section 3.3.1.

### 3.1.5. Correlation Analysis

First, we conducted a correlation analysis of each variable with respect to RUL to determine which features were potentially highly influential in resulting in variations in RUL.

FD001 - Feature correlation with RUL (sorted by |corr|):

| | corr_with_RUL | abs_corr_with_RUL |
|---|---|---|
| s11 | -0.775230 | 0.775230 |
| s4 | -0.757157 | 0.757157 |
| s12 | 0.748870 | 0.748870 |
| s7 | 0.733021 | 0.733021 |
| s15 | -0.720858 | 0.720858 |
| s21 | 0.707334 | 0.707334 |
| s20 | 0.704626 | 0.704626 |
| s17 | -0.680829 | 0.680829 |
| s2 | -0.678458 | 0.678458 |
| s3 | -0.655030 | 0.655030 |
| s8 | -0.624568 | 0.624568 |
| s13 | -0.624034 | 0.624034 |
| s9 | -0.462151 | 0.462151 |
| s14 | -0.369753 | 0.369753 |
| s6 | -0.108289 | 0.108289 |
| setting2 | -0.007091 | 0.007091 |
| setting1 | -0.005556 | 0.005556 |

*Fig 3.1.5: Correlation of features with RUL for FD001*

From the above analysis, we can observe that there are many variables in FD001 have a high correlation to RUL.

Next, we wanted to find which features had a high correlation with one another to identify potential features that may result in overfitting of our dataset. This would help us in feature selection down the line to ensure as much independence between features as possible.

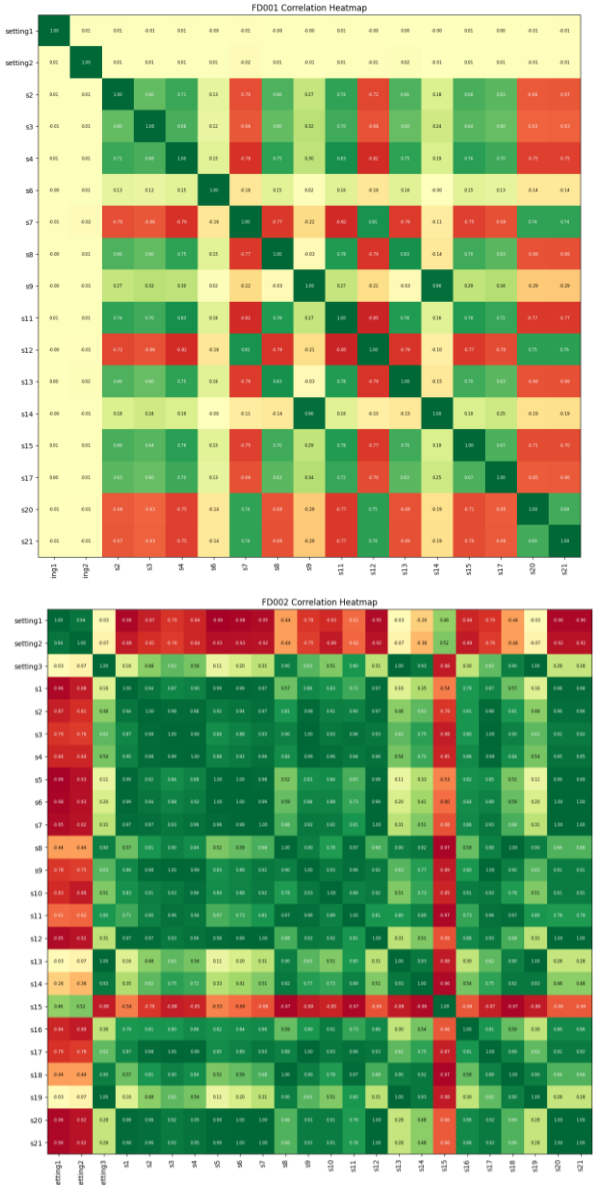



*Fig 3.1.5: Interfeature Correlation Analysis for FD001*

From the correlation heatmap above where green represents highly positive correlations and red represents highly negative correlations, we can identify pairs of features in FD001 such as s14 and s9 as well as s12 and s11 (>0.85 correlation). We can use this knowledge on top of the RUL correlation analysis to help determine which features to remove in Section 3.3.2.

## 3.2. Data Preprocessing

### 3.2.1. RUL Label Reconstruction

Rather than using a raw linear countdown from maximum cycle to zero, RUL can be reconstructed as RUL = min(max_cycle – current_cycle, 125). This collpases all early-life labels that would exceed 125 down to a flat ceiling of 125. As discussed in Section 3.1.3, engines do not degreade the moment they are switched on. Degradation signals emerge meaningfully only in the last cycles of an engine's life. This would mean that for example an engine at cycle 5 with lifespan of 300 cycles (RUL = 295) would have near identical sensor readings to another equivalent engine at cycle 50 (RUL = 250). Models trained on this will be forced to learn a distinction between RULs that does not exist within input features. A piecewise cap eliminates this by treating any engine beyond 125 cycles as just a state of "healthy" to ensure uniform treatment across all engines in the earlier phases.

The above effect of capping at 125 ensures that engines at healthy plateaus receive same label. This overall helps the models learn faster in degradation regions which contains most of the feature variations and reducing training capacity wasted on high RUL values with similar sensor readings.

An additional motivation to choosing RUL cap at 125 would be that it is the standard adopted by the vast majority of published CMAPSS benchmarks, which allows our RMSE to be computed on the same label space as other papers (Heimes, 2008). This ensures that the model results are more comparable against SOTA equivalents.

### 3.2.2. Min-max Normalisation of Features

For each numerical feature column (all settings and sensor readings), we apply a min-max normalisation, which maps every feature into the range [0,1]. Critically, the min and max are computed only from the training set and applied to both the training and test sets since test set statistics are unseen and normalisation can only be applied using seen data.

The purpose of min-max normalisation is to remove scale dominance since every feature has their own scale. As seen in Figure 3.1.1.3, an example of difference in scale would be features like s9 have a magnitude in the thousands whereas features like s10 has a small spread around 1.

Without normalisation, features with larger absolute values would numberically dominate the gradient-based learning. This means that the neural network weight updates being proportional to the input signal, sensor with values with higher scale would receive disproportionately large gradient updates compared to a sensor with a smaller scale.

Additionally, by normalising the features, each feature is now more comparable in magnitude as differences in normalised features are more directly comparable. Gradient descent would be able to converge faster as normalisation reshapes loss landscape towards a smoother geometry as a result.

## 3.3. Feature Engineering

### 3.3.1. Low-Variance Feature Removal

As discussed in Section 3.1.4, there exists a significant number of features in FD001 that have zero variance. For FD001 specifically, there are 7 features that are dropped (s1, s5, s10, s16, s18, s19, setting3) as they are constant by design of the operational condition set in FD001.

Low variance feature removal helps to elimiminate constant features that have zero contribution towards variations in RUL. Hence, this means that no matter what coefficient is assigned to these features, the feature would then contribute equally to any prediction, rendering it unworthy to keep. The key benefits of this for our project purposes would be to ensure efficiency in computation by removing the dimensionality of the input vectors, which would improve training speeds significantly.

### 3.3.2. Feature Pruning Using RUL and Interfeature Correlation

From Section 3.1.5, we have determined that variables s14 and s9 and s12 and s11 show high correlation with one another. To decide which one to prune exactly, we also refer to the RUL correlation analysis. Hence, we decide to prune s14 instead of s9 because s9 (-0.462151) shows a higher correlation to RUL compared to s14 (-0.369753). Between s12 and s11, both have relatively high correlations with RUL (~0.75 each) which indicates to us that we should experiment between dropping s12 and s11 rather than immediately drop one or the other.

## 3.4. Data Splitting, Model Saving, and Loading

To ensure robust model training and evaluation, a systematic approach was employed for data splitting, model checkpointing, and loading. This section details these critical processes.

The dataset is split into training, validation, and test sets to facilitate model development and unbiased performance evaluation. A key aspect of our strategy is to ensure that engines are not mixed between training and validation sets, preventing data leakage and providing a more realistic assessment of the model's generalization capabilities.

1. **Train-Validation Split**: The full training dataset is first split into training and validation sets. This split is performed at the **engine ID level** to ensure that all cycles for a given engine are either entirely in the training set or entirely in the validation set. Specifically, 80% of the unique engine IDs are allocated to the training set, and the remaining 20% are allocated to the validation set. This is achieved by:

    a. Identifying all unique engine IDs.
    b. Shuffling these unique IDs using a fixed random seed (np.random.default_rng(seed=42)) for reproducibility.
    c. Dividing the shuffled IDs into 80% for training and 20% for validation.
    d. Filtering the original training set to create train and val based on these allocated engine IDs.

2. **Test Set**: The dedicated test dataset is used exclusively for final model evaluation and is kept separate throughout the training and validation phases.

3. **Sequence Creation**: For each of the dataset dataframes, time-series sequences are created using a sliding window approach. This function transforms the engine's operational data into sequences of a predefined length (seq_len) and corresponding RUL labels.

4. **DataLoader Preparation**: The generated sequences (X_train, y_train, etc.) are then converted into TensorDataset objects and wrapped in DataLoaders. The train_loader is shuffled, while val_loader and test_loader are not, to maintain sequence order for evaluation. The seed_worker function is used during DataLoader initialization to ensure reproducibility of data loading.

During the training process, the model's performance on the validation set is continuously monitored. To prevent overfitting and ensure that the best performing model is retained, a checkpointing mechanism is implemented:

- The **Root Mean Squared Error (RMSE)** on the validation set is calculated at the end of each epoch.
- If the current val_rmse is lower than the best_val_rmse recorded so far, the current model's state dictionary (model.state_dict()) is saved to a specified model_save_path using torch.save().
- This ensures that only the model weights corresponding to the best validation performance are preserved.

After the training loop is complete, the best-performing model (saved during training) is loaded back into the model architecture for final evaluation on the test set. This ensures that the evaluation is performed on the model that demonstrates the best generalization capability on unseen validation data.

## 3.5. Model Architectures

### 3.5.1. Recurrent Neural Networks (RNN)

#### 3.5.1.1. RNN

The first model employed is a two-layer RNN designed for RUL regression. The overview of our RNN architecture is as follows:

**RNN Layer 1:** Takes an input of size $d_{\text{in}}$ (number of sensor features) and maps it to a hidden representation of size $h_1 = 100$, using a $\tanh$ nonlinearity with batch-first input ordering.

- **Dropout Layer 1:** Applies stochastic regularisation with dropout rate $p = 0.2$ to the full output sequence of Layer 1
- **RNN Layer 2:** Receives the regularised sequence of size $h_1 = 100$ and produces a hidden representation of size $h_2 = 50$, again using $\tanh$ nonlinearity
- **Dropout Layer 2:** Applies dropout at $p = 0.2$ to the output sequence of Layer 2
- **Temporal Pooling:** Extracts only the final time step's hidden state, which contains the encoding the full degradation history up to the last observed cycle
- **Fully Connected Output Layer:** A linear projection of the hidden state to a scalar prediction of RUL.
- **Loss Function:** Mean Squared Error (predicted RUL – target RUL) squared

We use a vanilla RNN as our natural baseline for time-series data. Engine sensor readings are inherently ordered which makes RNN suitable due to its ability to capture temporal patterns (from beginning to end) through its hidden state.

We use a two-layer stacked design to allow the network to learn abstractions. First layer focuses on capturing low-level sensor relationships and the second layer composes these relationships to higher level degradation trends. The tanh activation used keeps hidden state values bounded to ensure numerical stability which is important for regression tasks as it ensures the hidden state fed into the linear layer would make the linear layer's job more predictable. We also use dropout between layers to mitigate overfitting.

#### 3.5.1.2. GRU

The second model is a two-layer Gated Recurrent Unit, sharing the same overall structure as the baseline RNN but just replacing the vanilla recurrent cells with GRU cells. Otherwise, the architecture is structurally identical for consistency purposes.

- **GRU Layer 1:** Takes input of size $d_{\text{in}}$ and maps to hidden size $h_1 = 100$, with batch-first ordering
- **Dropout Layer 1:** Stochastic regularization at $p = 0.2$ applied across the full output sequence
- **GRU Layer 2:** Receives the regularized sequence of size $h_1 = 100$ and produces hidden representations of size $h_2 = 50$
- **Dropout Layer 2:** Dropout at $p = 0.2$ applied to the output sequence of Layer 2
- **Temporal Pooling:** Final time step hidden state is extracted
- **Fully Connected Output Layer:** Linear projection of hidden state to scalar prediction of RUL
- **Loss Function:** MSE

Engine degradation in the FD001 dataset unfolds over hundreds of operational cycles, which means that models must retain relevant information across many cycles. Vanilla RNNs struggle here as their gradients diminish exponentially as they backpropagate through many time steps, which cause earlier cycle sensor patterns to become negligible on the final prediction. In contrast, GRU's gate mitigates this by selectively preserving important hidden state information across many cycles such as memory of an early anomalous sensor reading that may be a key predictor of failure. Compared to more complex LSTM, GRU achieves similar memory capacity with fewer parameters (no cell state, output gate), which makes it computationally efficient and choice for a smaller FD001 dataset.

#### 3.5.1.3. LSTM

The third model is a two-layer LSTM network. It follows the same structural template as the RNN and GRU models but simply uses LSTM cells instead.

- **LSTM Layer 1:** Takes input of size $d_{\text{in}}$ and maps to hidden size $h_1 = 100$, with batch-first ordering
- **Dropout Layer 1:** Stochastic regularization at $p = 0.2$ applied across the full output sequence
- **LSTM Layer 2:** Receives the regularized sequence of size $h_1 = 100$ and produces hidden representations of size $h_2 = 50$
- **Dropout Layer 2:** Dropout at $p = 0.2$ applied to the output sequence of Layer 2
- **Temporal Pooling:** Final time step hidden state is extracted
- **Fully Connected Output Layer:** Linear projection of hidden state to scalar predictions of RUL
- **Loss Function:** MSE

LSTM introduces a dedicated cell state which is a separate memory highway alongside the hidden state. It is governed by the three gates (forget, input and output gate). This decoupling of the cell and hidden state gives LSTM finer-grained control as the cell state can carry long-range information largely intact while hidden state only exposes what is relevant at each cycle.

The LSTM's explicit cell state and three-gate mechanism make it the most capable of the RNN architectures discussed so far in terms of capturing long-range temporal dependencies. The trade-off relative to the GRU is a higher parameter count, which results in the LSTM being the most data-hungry of the three baselines. Its comparison against the GRU on FD001, a relatively smaller size CMAPSS dataset, would directly help us assess whether this additional cell state complexity would actually yield meaningful gains.

### 3.5.2. Transformer

To capture the complex temporal degradation patterns of aircraft engines, we explored the use of Transformer-based models (defined as RULTransformer), a sequence-to-one

regression model implemented in PyTorch. This model is designed as a sequence-to-one architecture that leverages self-attention mechanisms to capture long-range dependencies within sensor data. The overview of our RULTransformer architecture is as follows:

1. **Input Linear Layer:** Takes an input of size d_in (number of sensor features) and projects it to a higher-dimensional latent representation of size d_model = 64.

2. **Learnable Positional Encoding:** Injects sequence order information by adding a learnable parameter of shape [1, seq_len, d_model], initialized with a small normal distribution ($\mu$=0, $\sigma$=0.02) provides the multi-head self-attention mechanism with immediate, distinct signals to differentiate individual time steps from the first epoch, significantly accelerating the model's early convergence rate.

3. **Transformer Encoder Stack:** Consists of N stacked layers (where N is a tunable parameter, default N=2), each utilizing 4 attention heads (n_head=4) and a dropout rate of p=0.1 with batch-first input ordering.

4. **Temporal Pooling:** Extracts only the final time step's hidden state from the encoder output, which acts as a summary vector of the entire engine degradation history up to the current cycle.

5. **Fully Connected Regressor Head:** A multi-layer perceptron that first reduces the hidden state from 64 to 32 neurons with a ReLU nonlinearity and p=0.1 dropout, followed by a final linear projection to a scalar prediction of RUL.

6. **Loss Function:** MSE

Unlike RNNs, which must propagate information sequentially through a chain of hidden states resulting in an O(T) path length between distant time steps, the self-attention mechanism connects any two positions in the sequence directly in O(1) path length, making it significantly more robust at capturing long-range dependencies without the risk of vanishing gradients.

We use a learnable positional encoding instead of fixed sinusoids to allow the model to adaptively learn temporal representations suited to the specific degradation patterns present in the dataset, rather than imposing a generic fixed prior that may not reflect the structure of engine health trajectories.

The Transformer Encoder stack enables the network to learn hierarchical abstractions, where lower layers capture local sensor interactions and higher layers synthesise these into global health indicators. The inclusion of ReLU and dropout in the regressor head ensures the model can capture non-linear relationships while maintaining robustness against overfitting.

### 3.5.3. Hybrid

Three hybrid models namely LSTM-Transformer, GRU-Transformer, and CNN-Transformer are used to address a time-series forecasting problem which requires both short-term feature extraction and long-range context achieved by using the attention mechanism. Each model combines a front-end sequence encoder with a Transformer encoder and a final regression head, allowing the network to capture degradation patterns from sensor measurements more effectively than a single architecture alone.

#### 3.5.3.1. LSTM-Transformer

The first hybrid model combines a two-layer *Long Short-Term Memory* network with a *Transformer encoder*. The two components are compliment each other to achieve the best result. The LSTM stage is responsible for learning sequential degradation dynamics from multivariate engine sensor inputs, while the Transformer stage refines these temporal representations by modeling interactions across the full sequence. This design is suitable for RUL prediction because engine degradation often depends on both local temporal trends and longer-range dependencies.

- **LSTM Layer 1**: Takes input features of size num_features and produces hidden representations of size lstm_hidden = 64, using batch-first ordering.
- **Dropout**: Applied between stacked LSTM layers with rate 0.1 when more than one layer is used.
- **LSTM Layer 2**: Receives the hidden sequence from the previous layer and outputs a sequence of the same hidden dimension.
- **Linear Projection**: Maps the LSTM hidden dimension from 64 to Transformer embedding size d_model = 64.

- **Transformer Encoder**: Consists of 2 encoder layers, each with 4 attention heads and feedforward dimension 256.
- **Temporal Pooling**: The hidden state from the final time step is selected as the sequence summary.
- **Fully Connected Head**: A two-layer regression head maps the Transformer output to a single RUL value.
- **Loss Function**: Mean Squared Error (MSE).

#### 3.5.3.2. GRU-Transformer

The second hybrid model shares the same overall structure as the LSTM-Transformer to ensure architectural consistency, but uses GRU cells instead of LSTM cells. The GRU is computationally lighter than the LSTM while still capturing temporal dependencies.

- **GRU Layer 1**: Takes input of size num_features and maps it to hidden size gru_hidden = 64, with batch-first ordering.
- **Dropout**: Applied between stacked GRU layers with rate 0.1 when more than one layer is used.
- **GRU Layer 2**: Processes the sequence output of the first GRU layer and produces hidden representations of size 64.
- **Linear Projection**: Projects GRU outputs to d_model = 64 for Transformer compatibility.
- **Transformer Encoder**: Uses 2 encoder layers with 4 attention heads and feedforward dimension 256.
- **Temporal Pooling**: The last time-step representation is extracted as the sequence summary.
- **Fully Connected Head**: A regression block with one hidden layer of size 128, ReLU activation, dropout, and a final linear layer.

- **Loss Function**: Mean Squared Error (MSE).

#### 3.5.3.3. CNN-Transformer

The last hybrid model uses one-dimensional convolutional feature extractor. Instead of processing the sequence one by one, the CNN can learn the local temporal patterns such as short-term fluctuations, abrupt changes, and sensor-level abnormally. The extracyed features are then passed into the Transformer encoder to capture context across the sequence.

- **1D Convolution Layer:** Takes num_features input channels and produces num_filters = 32 feature maps using kernel size 3 with same-padding.
- **ReLU Activation:** Introduces nonlinearity after convolution.
- **Batch Normalization:** Stabilizes feature distribution and improves training robustness.
- **Dropout:** Applied with rate 0.1 to reduce overfitting.
- **Linear Projection:** Maps the convolutional feature dimension from 32 to Transformer embedding size 64.
- **Transformer Encoder:** Contains 2 encoder layers with 4 attention heads and feedforward dimension 256.
- **Temporal Pooling:** The final time-step Transformer output is used as the sequence representation.
- **Fully Connected Head:** A two-layer regression head outputs a scalar RUL estimate.
- **Loss Function:** Mean Squared Error (MSE).

## 3.6. Reproducibility

```python
def seed_everything(seed=1234):
    os.environ["PYTHONHASHSEED"] = str(seed)
    os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":16:8"

    random.seed(seed)
    np.random.seed(seed)

    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    torch.use_deterministic_algorithms(True)
```

*Fig 3.5: Seeding Function for Reproducibility*

- torch.manual_seed ensures identical CPU-side weight initialization and dropout masks across runs
- np.random.seed fixes any NumPy-based data preprocessing or shuffling
- random.seed controls Python-level randomness such as dataset splits
- torch.cuda.manual_seed_all ensures GPU-side operations produce the same results across runs

As each library maintains its own internal Pseudo-Random Number Generator (PRNG) state, seeding all four together guarantees that the model starts from the same initial weights, sees data in the same order, and applies the same dropout patterns on every run, making results directly comparable and independently reproducible.

To run the provided code and reproduce the results, the following Python packages are the core requirements:

| Package | Purpose |
|---|---|

| | |
|---|---|
| pandas | Data manipulation and analysis |
| numpy | Numerical operations |
| torch | Deep learning framework (PyTorch) |
| torch.nn | Neural network modules |
| torch.utils.data | Data loading utilities (DataLoader, TensorDataset) |
| sklearn.preprocessing | Data scaling (MinMaxScaler) |
| sklearn.metrics | Performance metrics (mean_squared_error) |
| math | Mathematical functions |
| os | Operating system interaction |
| random | Python's built-in random module |

The full list of requirements can be found in the requirements.txt file in our GitHub repository.

## 4. Malfunction

While training the models, we realised that the value set for the Seed may change the results (RMSE) dramatically depending on the model. For example, when training a LSTM model, the Test RMSE can vary from 23.43 (Seed=42) to 57.17 (Seed=1234), causing 'malfunction'.

| | Sequence Length | 50 | | | | | |
|---|---|---|---|---|---|---|---|
| | Models | GRU | | | LSTM | | |
| Dataset | Seed | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE |
| FD001_linear_rul_no_norm_0 | 42 | 57.1433 | 57.785 | 42.7017 | 57.1431 | 57.785 | 42.712 |
| FD001_linear_rul_no_norm_0 | 1234 | 57.1759 | 56.6008 | 42.2878 | 57.1674 | 56.6005 | 42.353 |
| FD001_linear_rul_no_norm_0 | 999 | 57.1446 | 58.7451 | 42.657 | 57.1481 | 58.7566 | 42.5785 |
| FD001_linear_rul_no_norm_0 | Mean | 57.1546 | 57.7103 | 42.54883 | 57.15287 | 57.71403 | 42.54783 |
| FD001_linear_rul_1 | 42 | 21.7667 | 22.0821 | 24.1052 | 23.4334 | 23.4524 | 23.8384 |
| FD001_linear_rul_1 | 1234 | 24.0107 | 23.4102 | 21.303 | 57.1724 | 56.6005 | 42.3133 |
| FD001_linear_rul_1 | 999 | 23.3619 | 24.2572 | 21.8105 | 57.1482 | 58.7569 | 42.5766 |
| FD001_linear_rul_1 | Mean | 23.04643 | 23.24983 | 22.40623 | 45.918 | 46.26993 | 36.24277 |
| FD001_piecewise_rul_2 | 42 | 10.5972 | 10.4198 | 14.3786 | 41.3615 | 41.573 | 40.0648 |
| FD001_piecewise_rul_2 | 1234 | 41.3774 | 41.6514 | 40.0138 | 41.3779 | 41.6514 | 40.0127 |
| FD001_piecewise_rul_2 | 999 | 8.3692 | 8.4677 | 14.5316 | 10.8848 | 10.7415 | 13.8462 |

*Figure 4. Effect of Different Seeds on RNN Training*

To mitigate the effects of the Seed number in our best model, we chose to vary it along with three hyperparameters when optimising for the best model.

## 5. Evaluation

All models are evaluated using RMSE against the test set for FD001 for comparison. The chosen test set would be on the final feature engineered version of FD001 (RUL cap of 125, min-max normalisation, low-variance features removed). While intermediate evaluation uses RMSE for model selection, the ultimate objective is anomaly detection (classifying engines as healthy or in a danger zone) which is addressed in Section 7. Our first objective would be to determine which dataset resulted in the best performing models to choose a final comparison dataset. The FD001 datasets initially tested in sequence was:

1. Raw unnormalised linear RUL (no RUL cap) dataset
   (FD001_linear_rul_no_norm_0 / dc1_0)
2. Normalised linear RUL dataset
   (FD001_linear_rul_1 / dc1_1)
3. Normalised RUL-capped at 125 dataset
   (FD001_piecewise_rul_2 / dc1_2)
4. Normalised RUL-capped AND low variance features removed dataset
   (FD001_low_variance_1 / fe2_lv1)
5. Normalised RUL-capped, low varianced removed, s14 removed
   (FD001_drop_s14 / fe2_hc2_1)
6. Normalised RUL-capped, low varianced removed, s14 and s11 removed

   (FD001_drop_s14_s11 / fe2_hc2_2)
7. Normalised RUL-capped, low varianced removed, s14 and s12 removed

   (FD001_ drop_s14_s12 / fe2_hc2_3)

Before fine-tuning, each architecture stuck to baseline default hyperparameters such as a sequence length of 50 for the datasets we would use for training and testing and a learning rate of 0.001.

For hyperparameter tuning, each architecture would then vary the sequence length across [30,50,70], learning rates across [0.001, 0.0005, 0.0001].

## 5.1. RNN-Based Training Results and Selection of Final Comparison Dataset

To determine which dataset of the seven listed should be the one that we decide on for model comparisons, we first conduct our preliminary testing for all datasets cross multiple hyperparameters on the RNN architecture first since it is the simplest and quickest architecture for training runs.

More importantly, we acknowledge that results may often be skewed towards unlucky or lucky seeding from experimentation and thus, we run the 7 datasets across the 3 RNN model architectures across 3 seeds (42, 999, 1234 chosen arbitrarily) for training and testing. We take the average RMSE values across multiple seeds to average out the effect of lucky or unlucky seeds across models.

For further consistency, we also conduct hyperparameter tuning for the RNN architecture to simultaneously select the best RNN architecture model and the final comparison dataset through consistency. The hyperparameters that were tuned were sequence lengths of the dataset (30,50,70) and learning rate (1e-3, 5e-4, 1e-4). Additionally, we implement patience for early stopping on validation performance, so training halts when the validation RMSE does not improve for 5 consecutive epochs, which helps reduce overfitting and avoids unnecessary training time while retaining the best model checkpoint. The code for training the RNN-based models and plotting the loss curve is found at /RNN-based/tuning. Code to reproduce the best model's inference is found at /RNN-based/best_model_and_evaluation.

| | Sequence Length | 30 | | | | | | | | | 50 | | | | | | | | | 70 | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Models | RNN | | | GRU | | | LSTM | | | RNN | | | GRU | | | LSTM | | | RNN | | | GRU | | | LSTM | | |
| Dataset | Seed | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE |
| FD001_linear_rul_no_norm_0 | 42 | 61.6896 | 61.1526 | 45.1676 | 61.6896 | 61.1526 | 45.1646 | 61.6897 | 61.1526 | 45.1612 | 57.1431 | 57.785 | 42.7109 | 57.1433 | 57.785 | 42.7017 | 57.1431 | 57.785 | 42.712 | 52.8603 | 53.7313 | 41.308 | 52.8602 | 53.7313 | 41.3093 | 52.8603 | 53.7313 | 41.3076 |
| FD001_linear_rul_no_norm_0 | 1234 | 61.7001 | 61.2308 | 44.8726 | 61.6923 | 61.2312 | 45.0603 | 61.6961 | 61.2305 | 44.9575 | 57.1802 | 56.6014 | 42.2586 | 57.1759 | 56.6008 | 42.2878 | 57.1674 | 56.6005 | 42.353 | 52.8612 | 53.8889 | 41.2885 | 52.8625 | 53.8931 | 41.2698 | 52.8616 | 53.8904 | 41.2817 |
| FD001_linear_rul_no_norm_0 | 999 | 61.6904 | 63.3657 | 45.1289 | 61.6911 | 63.3676 | 45.1016 | 61.6904 | 63.3655 | 45.1311 | 57.1462 | 58.751 | 42.6154 | 57.1446 | 58.7451 | 42.657 | 57.1481 | 58.7566 | 42.5785 | 52.8608 | 52.678 | 41.2958 | 52.8612 | 52.679 | 41.2895 | 52.8616 | 52.68 | 41.2829 |
| FD001_linear_rul_no_norm_0 | Mean | 61.69337 | 61.91637 | 45.05637 | 61.691 | 61.91713 | 45.1083 | 61.69207 | 61.9162 | 45.08327 | 57.1565 | 57.71247 | 42.5283 | 57.1546 | 57.7103 | 42.54883 | 57.15287 | 57.71403 | 42.54763 | 52.86977 | 53.43273 | 41.29743 | 52.8613 | 53.43447 | 41.28963 | 52.86117 | 53.4339 | 41.29073 |
| FD001_linear_rul_1 | 42 | 61.6897 | 61.1526 | 45.164 | 26.8304 | 27.9541 | 25.2704 | 61.6897 | 61.1526 | 45.1603 | 57.143 | 57.785 | 42.7142 | 21.7667 | 22.0821 | 24.1052 | 23.4334 | 23.4524 | 23.8384 | 52.8603 | 53.7313 | 41.308 | 16.1069 | 16.714 | 21.8549 | 18.5449 | 19.2627 | 20.3478 |
| FD001_linear_rul_1 | 1234 | 61.6991 | 61.2307 | 44.8928 | 30.8491 | 30.816 | 22.2192 | 61.6979 | 61.2305 | 44.9183 | 57.1771 | 56.6009 | 42.2794 | 24.0107 | 23.4102 | 21.303 | 57.1724 | 56.6005 | 42.3133 | 52.8612 | 53.8888 | 41.2889 | 17.4025 | 18.2398 | 22.0878 | 52.8617 | 53.8905 | 41.2811 |
| FD001_linear_rul_1 | 999 | 61.6904 | 63.3657 | 45.1291 | 29.5754 | 30.8931 | 24.2392 | 61.6904 | 63.3656 | 45.1299 | 57.1462 | 58.751 | 42.6154 | 23.3619 | 24.2572 | 21.8105 | 57.1482 | 58.7569 | 42.5766 | 52.8608 | 52.678 | 41.296 | 16.1069 | 16.714 | 21.8549 | 52.8616 | 52.68 | 41.2826 |
| FD001_linear_rul_1 | Mean | 61.69307 | 61.91633 | 45.0619 | 29.08497 | 29.58773 | 23.9096 | 61.69267 | 61.91623 | 45.06957 | 57.15543 | 57.7123 | 42.53633 | 23.04543 | 23.24983 | 22.40623 | 45.918 | 46.26993 | 36.24277 | 52.86977 | 53.4327 | 41.29763 | 16.53877 | 17.2226 | 21.93253 | 41.42273 | 41.9444 | 34.30373 |
| FD001_piecewise_rul_2 | 42 | 41.7899 | 41.4892 | 40.4845 | 12.767 | 12.6758 | 13.3531 | 41.7901 | 41.9959 | 40.4784 | 41.3636 | 41.7048 | 40.0532 | 10.5972 | 10.4196 | 14.3786 | 41.3615 | 41.573 | 40.0646 | 40.4219 | 40.6343 | 39.5838 | 9.7998 | 9.9611 | 13.106 | 40.42 | 40.3524 | 39.565 |
| FD001_piecewise_rul_2 | 1234 | 41.7918 | 41.7495 | 40.4488 | 13.0252 | 12.9612 | 13.1673 | 41.7912 | 41.7496 | 40.4568 | 41.3788 | 41.6514 | 40.0111 | 41.3774 | 41.6514 | 40.0138 | 41.3779 | 41.6514 | 40.0127 | 40.4185 | 40.5443 | 39.5869 | 7.8474 | 7.9305 | 14.6196 | 41.3779 | 41.6514 | 40.0127 |
| FD001_piecewise_rul_2 | 999 | 41.79 | 41.9956 | 40.4822 | 12.8907 | 12.641 | 13.2304 | 41.7898 | 41.4892 | 40.4859 | 41.3609 | 41.5704 | 40.0728 | 8.3692 | 8.4677 | 14.5316 | 10.8848 | 10.7415 | 13.8462 | 40.4189 | 40.3492 | 39.5863 | 8.8034 | 9.1793 | 14.0349 | 40.4212 | 40.6343 | 39.5841 |
| FD001_piecewise_rul_2 | Mean | 41.79067 | 41.74477 | 40.47183 | 12.8943 | 12.826 | 13.25233 | 41.79037 | 41.7449 | 40.47357 | 41.36763 | 41.6422 | 40.0457 | 20.1146 | 20.17963 | 22.97467 | 31.20807 | 31.32197 | 31.3079 | 40.41977 | 40.50927 | 39.58567 | 8.616867 | 9.023633 | 13.92 | 40.73987 | 40.87937 | 39.72777 |
| FD001_low_variance_1 | 42 | 41.7901 | 41.4892 | 40.4782 | 12.8527 | 12.8443 | 13.2486 | 41.7896 | 41.4892 | 40.4966 | 41.3639 | 41.7048 | 40.0521 | 11.4437 | 11.265 | 13.1672 | 41.3624 | 41.7048 | 40.0598 | 40.4222 | 40.6343 | 39.5837 | 40.4217 | 40.6343 | 39.5839 | 40.4204 | 40.6343 | 39.5846 |
| FD001_low_variance_1 | 1234 | 41.7921 | 41.7495 | 40.4449 | 12.6853 | 12.6205 | 13.2208 | 41.7916 | 41.7496 | 40.4511 | 41.3804 | 41.6515 | 40.0083 | 41.3822 | 41.6517 | 40.0053 | 41.3759 | 41.6514 | 40.0167 | 40.419 | 40.5461 | 39.5861 | 10.5828 | 10.4449 | 12.9159 | 40.4208 | 40.5514 | 39.5844 |
| FD001_low_variance_1 | 999 | 41.7899 | 41.9956 | 40.4822 | 12.0829 | 12.2085 | 13.0339 | 41.7898 | 41.9954 | 40.4859 | 41.3602 | 41.5696 | 40.0755 | 10.9969 | 10.8444 | 13.9021 | 41.3595 | 41.5669 | 40.0853 | 40.42 | 40.3525 | 39.5849 | 9.9954 | 10.287 | 13.3189 | 40.4205 | 40.3538 | 39.5845 |
| FD001_low_variance | Mean | 41.7907 | 41.74477 | 40.46843 | 12.5403 | 12.55777 | 13.16777 | 41.79033 | 41.74473 | 40.47787 | 41.36817 | 41.64197 | 40.0453 | 21.27427 | 21.2537 | 22.3582 | 41.36587 | 41.64103 | 40.05393 | 40.4204 | 40.51097 | 39.5849 | 20.3333 | 20.4554 | 21.93967 | 40.42057 | 40.51317 | 39.5845 |
| FD001_drop_s14 | 42 | 41.7901 | 41.4893 | 40.4775 | 12.958 | 12.9404 | 13.2429 | 41.7901 | 41.4892 | 40.4794 | 41.364 | 41.7049 | 40.0514 | 41.3631 | 41.7048 | 40.056 | 41.3632 | 41.7048 | 40.0554 | 40.422 | 40.6343 | 39.5838 | 40.4209 | 40.6343 | 39.5843 | 40.4217 | 40.6343 | 39.5839 |
| FD001_drop_s14 | 1234 | 41.7918 | 41.7495 | 40.4494 | 12.8811 | 12.776 | 13.0796 | 41.7911 | 41.7497 | 40.4581 | 41.7918 | 41.7495 | 40.4494 | 41.3798 | 41.6515 | 40.0094 | 41.3782 | 41.6514 | 40.0122 | 40.4192 | 40.5467 | 39.5858 | 7.7618 | 7.823 | 15.2153 | 40.4198 | 40.5487 | 39.5853 |
| FD001_drop_s14 | 999 | 41.7901 | 41.995 | 40.4776 | 12.6871 | 12.6727 | 13.6197 | 41.7903 | 41.9963 | 40.4731 | 41.3598 | 41.5687 | 40.0787 | 11.0468 | 10.9787 | 13.8588 | 41.3598 | 41.5686 | 40.0789 | 40.4188 | 40.349 | 39.5864 | 10.328 | 10.5992 | 14.3993 | 40.4199 | 40.3523 | 39.585 |
| FD001_drop_s14 | Mean | 41.79067 | 41.74433 | 40.46817 | 12.84207 | 12.79637 | 13.31407 | 41.7505 | 41.74507 | 40.4702 | 41.5052 | 41.67437 | 40.19317 | 31.26323 | 31.445 | 31.30807 | 41.36707 | 41.6416 | 40.04883 | 40.42 | 40.51 | 39.58533 | 19.50357 | 19.6855 | 23.0663 | 40.42047 | 40.51177 | 39.58467 |
| FD001_drop_s14_s11 | 42 | 41.79 | 41.4892 | 40.4813 | 12.8162 | 12.8253 | 13.8452 | 41.7901 | 41.4892 | 40.4787 | 41.3629 | 41.7048 | 40.0568 | 11.0441 | 11.0558 | 13.8076 | 41.3631 | 41.7048 | 40.056 | 40.4215 | 40.6343 | 39.584 | 8.7833 | 9.1637 | 15.0994 | 40.4219 | 40.6343 | 39.5838 |
| FD001_drop_s14_s11 | 1234 | 41.7923 | 41.7496 | 40.4427 | 13.427 | 13.4004 | 13.3579 | 41.1043 | 41.1323 | 13.7219 | 41.3838 | 41.6519 | 40.0028 | 41.3856 | 41.6522 | 40.0002 | 41.3792 | 41.6514 | 40.0104 | 40.4204 | 40.5502 | 39.5846 | 8.5177 | 8.5688 | 15.8189 | 40.4211 | 40.5522 | 39.5842 |
| FD001_drop_s14_s11 | 999 | 41.7906 | 41.9968 | 40.4672 | 13.2483 | 13.2379 | 13.4448 | 41.7899 | 41.9956 | 40.4827 | 41.3601 | 41.5695 | 40.0759 | 10.3394 | 10.3679 | 14.4213 | 41.3603 | 41.5699 | 40.0743 | 40.4195 | 40.3514 | 39.5853 | 40.4219 | 40.6343 | 39.5838 | 13.033 | 13.1202 | 13.6119 |
| FD001_drop_s14_s11 | Mean | 41.79067 | 41.74517 | 40.46367 | 13.16383 | 13.15433 | 13.5463 | 32.2281 | 32.2057 | 31.56967 | 41.36893 | 41.64207 | 40.04517 | 20.92303 | 21.0253 | 22.74303 | 41.36753 | 41.64203 | 40.0469 | 40.4205 | 40.51197 | 39.58463 | 19.24097 | 19.4556 | 23.5067 | 31.292 | 31.43557 | 30.92663 |
| FD001_drop_s14_s12 | 42 | 41.79 | 41.4892 | 40.4813 | 12.6442 | 12.6296 | 13.7207 | 41.7901 | 41.4892 | 40.4787 | 41.3629 | 41.7048 | 40.0567 | 11.051 | 10.9743 | 14.4012 | 41.363 | 41.7048 | 40.0561 | 40.4215 | 40.6343 | 39.584 | 9.3633 | 9.6567 | 14.4212 | 40.4219 | 40.6343 | 39.5838 |
| FD001_drop_s14_s12 | 1234 | 41.7923 | 41.7495 | 40.4425 | 13.07 | 12.9399 | 13.3732 | 41.7927 | 41.7495 | 40.4389 | 41.3838 | 41.6519 | 40.0028 | 41.3855 | 41.6522 | 40.0003 | 41.3793 | 41.6514 | 40.0103 | 40.4204 | 40.5502 | 39.5846 | 6.955 | 7.0418 | 15.8255 | 40.4211 | 40.5522 | 39.5842 |
| FD001_drop_s14_s12 | 999 | 41.7906 | 41.9968 | 40.4672 | 12.9265 | 13.032 | 13.3233 | 13.4686 | 13.3472 | 13.6658 | 41.3601 | 41.5695 | 40.0759 | 9.9807 | 10.0944 | 14.9045 | 11.292 | 11.1988 | 13.3753 | 40.4196 | 40.3514 | 39.5853 | 7.5719 | 7.926 | 16.2663 | 11.1156 | 11.2617 | 13.076 |
| FD001_drop_s14_s12 | Mean | 41.79067 | 41.74517 | 40.46367 | 12.88023 | 12.86717 | 13.4724 | 32.35047 | 32.1953 | 31.5278 | 41.36893 | 41.64207 | 40.04513 | 20.80573 | 20.90667 | 23.102 | 31.34477 | 31.51833 | 31.14723 | 40.4205 | 40.51197 | 39.58463 | 7.9634 | 8.208167 | 15.50433 | 30.65287 | 30.81607 | 30.748 |

*Figure 5.1.1: Train, Val and Test RMSE for 7 datasets across [30,50,70] Sequence Lengths for RNN Architecture*

| | Learning Rate | 1e(-3) | | | | | | | | | 5e(-4) | | | | | | | | | 1e(-4) | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Models | RNN | | | GRU | | | LSTM | | | RNN | | | GRU | | | LSTM | | | RNN | | | GRU | | | LSTM | | |
| Dataset | Seed | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE | Train RMSE | Val RMSE | Test RMSE |
| FD001_linear_rul_no_norm_0 | 42 | 57.1431 | 57.785 | 42.7109 | 57.1433 | 57.785 | 42.7017 | 57.1431 | 57.785 | 42.712 | 57.1429 | 57.785 | 42.7182 | 57.1431 | 57.785 | 42.711 | 57.1431 | 57.785 | 42.7089 | 66.6768 | 67.059 | 46.6269 | 66.8721 | 67.2513 | 46.8069 | 48.5668 | 48.5582 | 45.5609 |
| FD001_linear_rul_no_norm_0 | 1234 | 57.1802 | 56.6014 | 42.2586 | 57.1759 | 56.6008 | 42.2878 | 57.1674 | 56.6005 | 42.353 | 57.1703 | 56.6005 | 42.3299 | 57.1663 | 56.6006 | 42.3622 | 57.1628 | 56.601 | 42.3932 | 66.8887 | 65.5154 | 46.8223 | 66.5484 | 65.2828 | 46.6003 | 48.8545 | 48.4543 | 45.8252 |
| FD001_linear_rul_no_norm_0 | 999 | 57.1462 | 58.751 | 42.6154 | 57.1446 | 58.7451 | 42.657 | 57.1481 | 58.7566 | 42.5785 | 57.1457 | 58.7524 | 42.606 | 57.1468 | 58.7518 | 42.6101 | 57.1466 | 58.7521 | 42.6079 | 67.0376 | 69.2397 | 46.9606 | 66.9171 | 69.1175 | 46.8486 | 48.5251 | 49.0224 | 45.5226 |
| FD001_linear_rul_no_norm_0 | Mean | 57.1565 | 57.71247 | 42.5283 | 57.1546 | 57.7103 | 42.54883 | 57.15287 | 57.71403 | 42.54763 | 57.1533 | 57.71263 | 42.55137 | 57.15197 | 57.71247 | 42.5611 | 57.15083 | 57.7127 | 42.57 | 66.8677 | 67.27137 | 46.80313 | 66.81253 | 67.2172 | 46.75193 | 48.6488 | 48.68163 | 45.63623 |
| FD001_linear_rul_1 | 42 | 57.143 | 57.785 | 42.7142 | 21.7667 | 22.0821 | 24.1052 | 23.4334 | 23.4524 | 23.8384 | 57.1679 | 56.6005 | 42.349 | 25.4291 | 25.3844 | 19.3022 | 27.057 | 26.9486 | 19.1054 | 66.737 | 65.3665 | 46.6819 | 66.7332 | 67.1145 | 46.6784 | 66.6211 | 67.0942 | 46.5751 |
| FD001_linear_rul_1 | 1234 | 57.1771 | 56.6009 | 42.2794 | 24.0107 | 23.4102 | 21.303 | 57.1724 | 56.6005 | 42.3133 | 57.1467 | 58.7524 | 42.6061 | 57.1661 | 56.6006 | 42.3643 | 57.1667 | 56.6006 | 42.3588 | 67.0235 | 69.2258 | 46.9478 | 66.619 | 65.2542 | 46.5731 | 66.7764 | 65.4066 | 46.7183 |
| FD001_linear_rul_1 | 999 | 57.1462 | 58.751 | 42.6154 | 23.3619 | 24.2572 | 21.8105 | 57.1482 | 58.7569 | 42.5766 | 57.1429 | 57.785 | 42.7186 | 26.2653 | 27.284 | 18.9617 | 30.0089 | 31.4468 | 16.8649 | 66.5946 | 66.9781 | 46.5508 | 66.8729 | 69.0727 | 46.8076 | 67.0834 | 69.2862 | 47.0032 |
| FD001_linear_rul_1 | Mean | 57.15543 | 57.7123 | 42.53633 | 23.04543 | 23.24983 | 22.40623 | 45.918 | 46.26993 | 36.24277 | 57.1525 | 57.71263 | 42.5579 | 36.28683 | 36.423 | 26.87607 | 38.07753 | 38.332 | 26.1097 | 66.78503 | 67.1507 | 46.72573 | 66.7417 | 67.14713 | 46.68637 | 66.82697 | 67.23233 | 46.76553 |
| FD001_piecewise_rul_2 | 42 | 41.3636 | 41.7048 | 40.0532 | 10.5972 | 10.4196 | 14.3786 | 41.3615 | 41.573 | 40.0648 | 41.3771 | 41.6514 | 40.0143 | 11.0388 | 10.8013 | 13.556 | 11.7299 | 11.5344 | 14.1392 | 48.6806 | 48.2974 | 45.6653 | 48.671 | 48.66 | 45.6565 | 48.5589 | 48.5504 | 45.5536 |
| FD001_piecewise_rul_2 | 1234 | 41.3788 | 41.6514 | 40.0111 | 41.3774 | 41.6514 | 40.0138 | 41.3779 | 41.6514 | 40.0127 | 41.3625 | 41.575 | 40.0592 | 41.3766 | 41.6513 | 40.0153 | 41.3761 | 41.6514 | 40.0164 | 48.9605 | 49.4638 | 45.9229 | 48.8159 | 48.4273 | 45.7896 | 48.7188 | 48.3341 | 45.7009 |
| FD001_piecewise_rul_2 | 999 | 41.3609 | 41.5704 | 40.0728 | 8.3692 | 8.4677 | 14.5316 | 10.8848 | 10.7415 | 13.8462 | 41.3622 | 41.7048 | 40.0609 | 11.221 | 11.2002 | 12.7336 | 41.3618 | 41.5735 | 40.0633 | 48.5327 | 48.5249 | 45.5297 | 48.8069 | 49.3081 | 45.7814 | 49.0093 | 49.5133 | 45.9679 |
| FD001_piecewise_rul_2 | Mean | 41.36763 | 41.6422 | 40.0457 | 20.1146 | 20.17963 | 22.97467 | 31.20807 | 31.32197 | 31.3079 | 41.36727 | 41.64373 | 40.0448 | 21.21213 | 21.2176 | 22.10163 | 31.48927 | 31.58643 | 31.4063 | 48.7246 | 48.76203 | 45.70587 | 48.7646 | 48.79847 | 45.74257 | 48.76233 | 48.79927 | 45.74067 |
| FD001_low_variance_1 | 42 | 41.3639 | 41.7048 | 40.0521 | 11.4437 | 11.265 | 13.1672 | 41.3624 | 41.7048 | 40.0598 | 41.3797 | 41.6515 | 40.0095 | 11.2157 | 10.9607 | 13.2962 | 41.3639 | 41.7049 | 40.0521 | 48.8281 | 48.4391 | 45.801 | 48.3015 | 48.2991 | 45.3179 | 48.2037 | 48.2039 | 45.2286 |
| FD001_low_variance_1 | 1234 | 41.3804 | 41.6515 | 40.0083 | 41.3822 | 41.6517 | 40.0053 | 41.3759 | 41.6514 | 40.0167 | 41.361 | 41.5718 | 40.0683 | 41.3783 | 41.6514 | 40.012 | 41.3727 | 41.6516 | 40.0237 | 48.6196 | 49.1182 | 45.6093 | 48.561 | 48.1826 | 45.5555 | 48.6364 | 49.1353 | 45.6248 |
| FD001_low_variance_1 | 999 | 41.3602 | 41.5696 | 40.0755 | 10.9969 | 10.8444 | 13.9021 | 41.3595 | 41.5669 | 40.0853 | 41.3627 | 41.7048 | 40.0576 | 10.9728 | 10.938 | 13.447 | 41.3609 | 41.5715 | 40.0692 | 49.0956 | 49.0751 | 46.0475 | 48.5272 | 49.0246 | 45.5246 | 48.6364 | 49.1353 | 45.6248 |
| FD001_low_variance | Mean | 41.36817 | 41.64157 | 40.0453 | 21.27427 | 21.2537 | 22.3582 | 41.36587 | 41.64103 | 40.05393 | 41.3678 | 41.6427 | 40.04513 | 21.18883 | 21.18337 | 22.25173 | 41.36583 | 41.64267 | 40.04833 | 48.84777 | 48.87747 | 45.81927 | 48.46323 | 48.5021 | 45.466 | 48.49217 | 48.8247 | 45.48273 |
| FD001_drop_s14 | 42 | 41.364 | 41.7049 | 40.0514 | 41.3631 | 41.7048 | 40.056 | 41.3632 | 41.7048 | 40.0554 | 41.3784 | 41.6514 | 40.0118 | 41.3626 | 41.7048 | 40.0584 | 41.3616 | 41.7049 | 40.0641 | 48.6136 | 48.6039 | 45.6039 | 48.5299 | 48.5221 | 45.527 | 48.5655 | 48.557 | 45.5597 |
| FD001_drop_s14 | 1234 | 41.7918 | 41.7495 | 40.4494 | 41.3798 | 41.6515 | 40.0094 | 41.3782 | 41.6514 | 40.0122 | 41.3613 | 41.5725 | 40.0663 | 41.4268 | 41.4291 | 12.6184 | 41.3751 | 41.6514 | 40.0183 | 48.8581 | 48.4678 | 45.8286 | 48.7348 | 48.3494 | 45.7152 | 48.4432 | 48.0697 | 45.4476 |
| FD001_drop_s14 | 999 | 41.3598 | 41.5687 | 40.0787 | 11.0468 | 10.9787 | 13.8588 | 41.3598 | 41.5686 | 40.0789 | 41.3623 | 41.7048 | 40.0599 | 41.3619 | 41.5738 | 40.0625 | 41.3615 | 41.5729 | 40.0649 | 48.559 | 49.0568 | 45.5537 | 48.4208 | 48.9157 | 45.4271 | 48.5232 | 49.0205 | 45.5209 |
| FD001_drop_s14 | Mean | 41.5052 | 41.67437 | 40.19317 | 31.26323 | 31.445 | 31.30807 | 41.36707 | 41.6416 | 40.04883 | 41.36733 | 41.6429 | 40.046 | 31.38377 | 31.56823 | 30.9131 | 41.36667 | 41.64307 | 40.0491 | 48.6768 | 48.7095 | 45.65297 | 48.56183 | 48.89667 | 45.55643 | 48.51063 | 48.54907 | 45.5094 |
| FD001_drop_s14_s11 | 42 | 41.3629 | 41.7048 | 40.0568 | 11.0441 | 11.0558 | 13.8076 | 41.3631 | 41.7048 | 40.056 | 41.377 | 41.6513 | 40.0145 | 11.2891 | 11.0509 | 13.9961 | 41.3776 | 41.6514 | 40.0134 | 48.7148 | 48.3302 | 45.6968 | 48.794 | 48.7802 | 45.7696 | 48.5635 | 48.555 | 45.5579 |
| FD001_drop_s14_s11 | 1234 | 41.3838 | 41.6519 | 40.0028 | 41.3856 | 41.6522 | 40.0002 | 41.3792 | 41.6514 | 40.0104 | 41.3614 | 41.5727 | 40.0655 | 11.3016 | 11.245 | 13.7835 | 41.3776 | 41.6514 | 40.0134 | 48.5398 | 49.0374 | 45.5362 | 48.9401 | 48.5466 | 45.9041 | 48.8521 | 48.462 | 45.823 |
| FD001_drop_s14_s11 | 999 | 41.3601 | 41.5695 | 40.0759 | 10.3394 | 10.3679 | 14.4213 | 41.3603 | 41.5699 | 40.0743 | 41.3629 | 41.7048 | 40.0566 | 11.313 | 11.285 | 13.4175 | 41.3614 | 41.5726 | 40.066 | 48.676 | 48.6549 | 45.6611 | 48.5548 | 49.0525 | 45.5499 | 48.5553 | 49.053 | 45.5503 |
| FD001_drop_s14_s11 | Mean | 41.36893 | 41.64207 | 40.04513 | 20.80573 | 21.0253 | 22.74303 | 41.36753 | 41.64203 | 40.0469 | 41.3671 | 41.6293 | 40.04553 | 11.30123 | 11.19833 | 13.73303 | 41.3722 | 41.62513 | 40.03093 | 48.64333 | 48.6775 | 45.63137 | 48.76297 | 48.7931 | 45.7412 | 48.65697 | 48.69 | 45.64373 |
| FD001_drop_s14_s12 | 42 | 41.3629 | 41.7048 | 40.0567 | 11.051 | 10.9743 | 14.4012 | 41.363 | 41.7048 | 40.0561 | 41.377 | 41.6513 | 40.0145 | 11.2554 | 11.0786 | 13.1946 | 41.3617 | 41.7049 | 40.0638 | 48.7124 | 48.328 | 45.6946 | 48.7901 | 48.7764 | 45.766 | 48.5668 | 48.5582 | 45.5609 |
| FD001_drop_s14_s12 | 1234 | 41.3838 | 41.6519 | 40.0028 | 41.3855 | 41.6522 | 40.0003 | 41.3793 | 41.6514 | 40.0103 | 41.3614 | 41.5727 | 40.0655 | 11.6906 | 11.6888 | 12.6316 | 41.3776 | 41.6514 | 40.0134 | 48.5416 | 49.0391 | 45.5377 | 48.937 | 48.5436 | 45.9012 | 48.8545 | 48.4543 | 45.8252 |
| FD001_drop_s14_s12 | 999 | 41.3601 | 41.5695 | 40.0759 | 9.9807 | 10.0944 | 14.9045 | 11.292 | 11.1988 | 13.3753 | 41.363 | 41.7048 | 40.0565 | 13.9072 | 14.0156 | 13.0064 | 13.1307 | 13.1486 | 13.678 | 48.6862 | 48.6749 | 45.6709 | 48.5556 | 49.0533 | 45.5506 | 48.5251 | 49.0224 | 45.5226 |
| FD001_drop_s14_s12 | Mean | 41.36893 | 41.64207 | 40.04513 | 20.80573 | 20.90667 | 23.102 | 31.34477 | 31.51833 | 31.14723 | 41.36713 | 41.64293 | 40.0455 | 12.2844 | 12.261 | 12.95087 | 31.95667 | 32.1683 | 31.25173 | 48.64673 | 48.68067 | 45.63427 | 48.7609 | 48.7911 | 45.73927 | 48.6488 | 48.68163 | 45.63623 |

*Figure 5.1.2: Train, Val and Test RMSE for 7 datasets across [1e-3, 5e-4, 1e-4] Learning Rates for RNN Architecture*

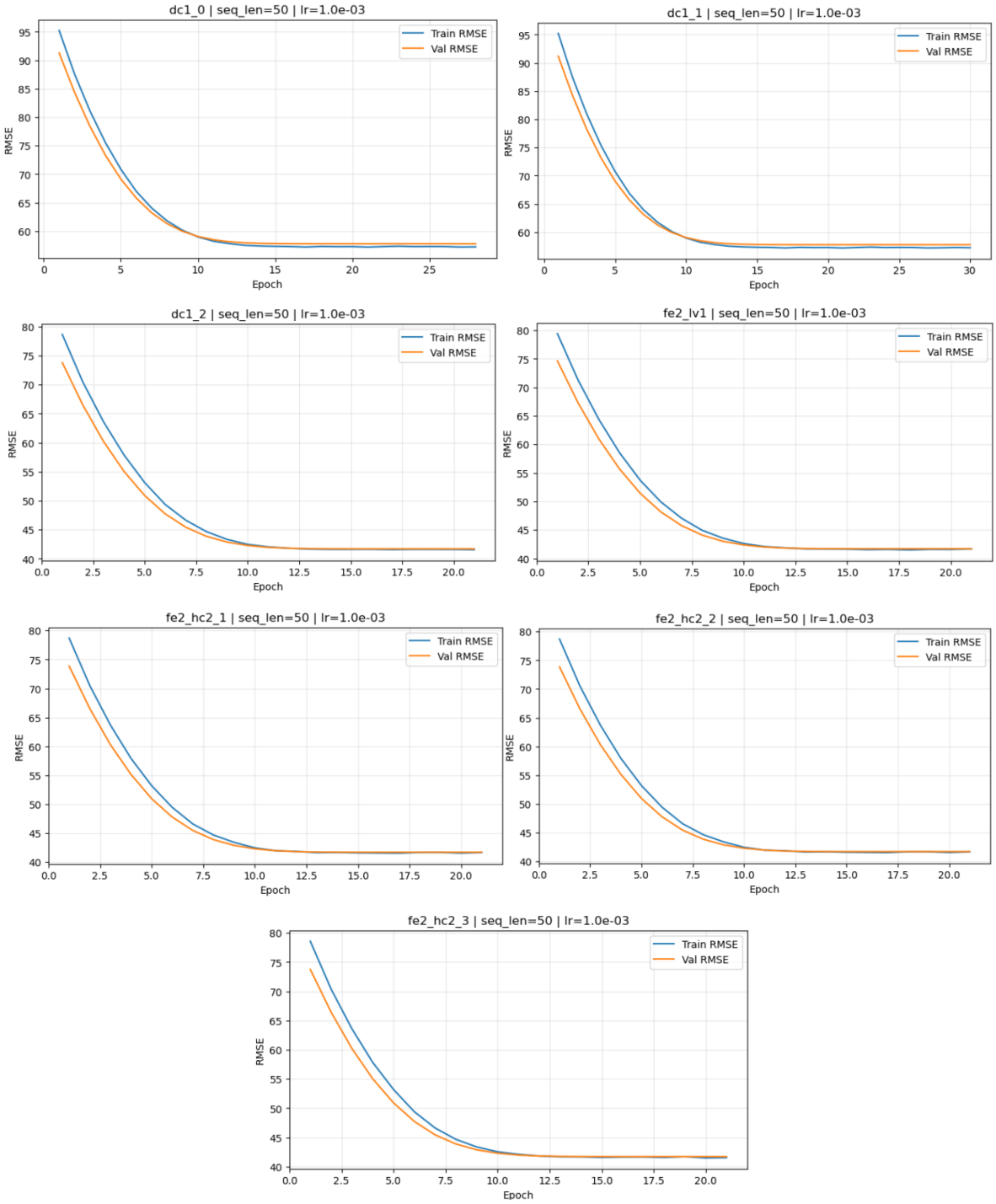



*Figure 5.1.3: Loss Curves for RNN Architecture (Seed 42) across 7 datasets at SEQ_LEN = 50, BATCH_SIZE = 64, EPOCHS = 50 and LEARNING_RATE = 1e-3 (SEQ_LEN=30, SEQ_LEN =70 not shown for brevity)*

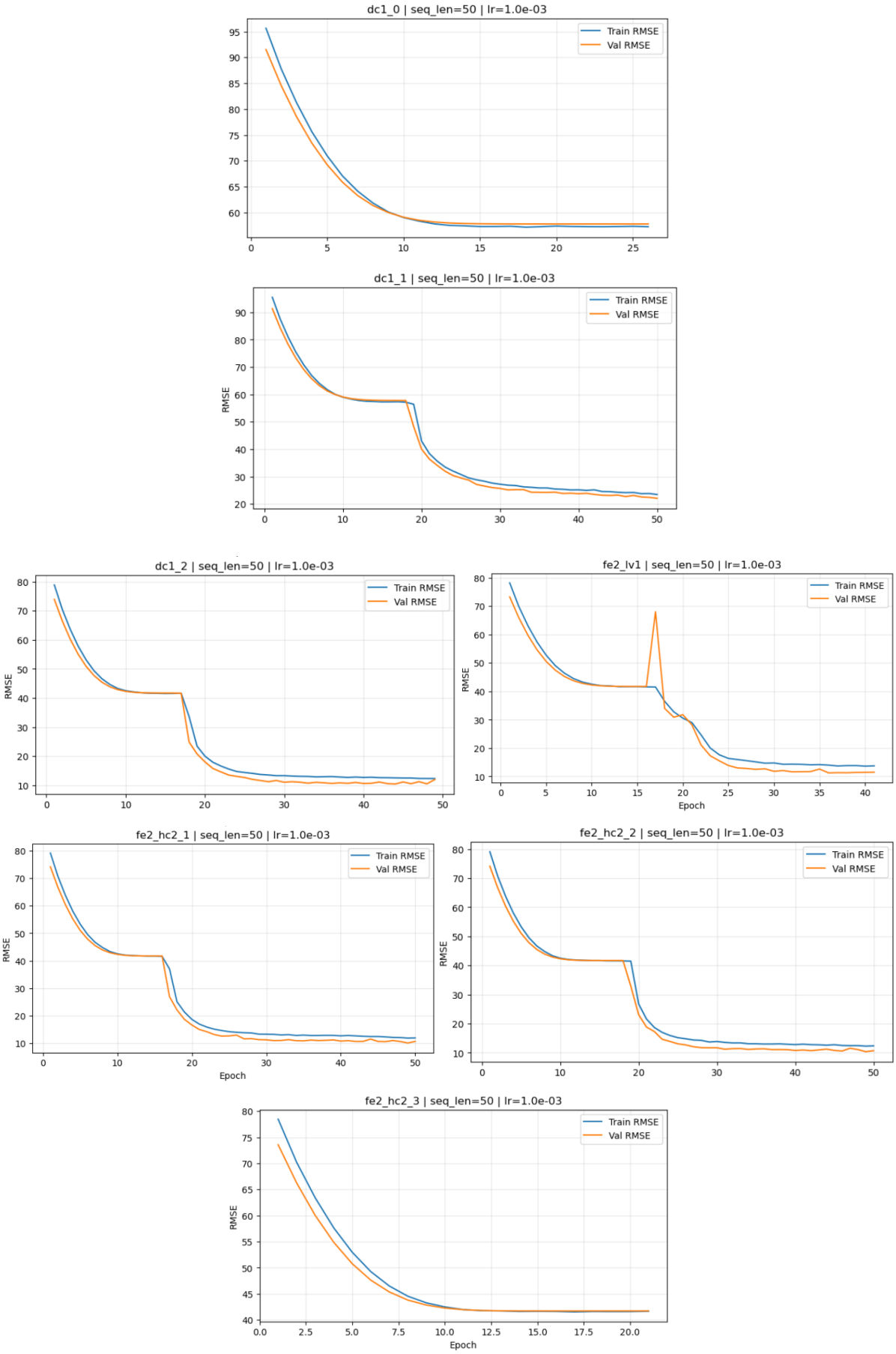
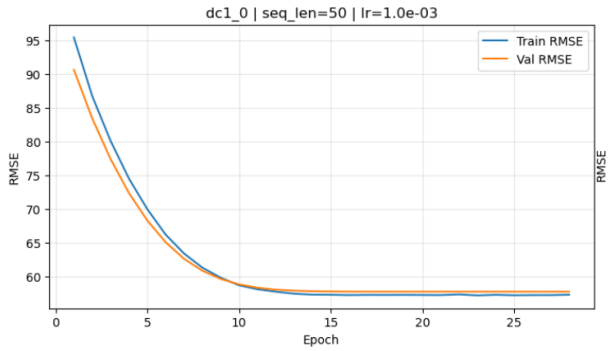
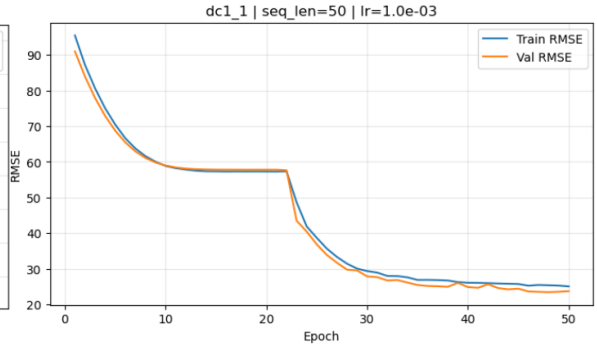
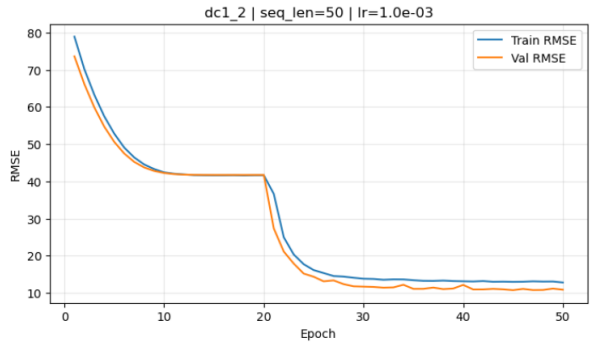


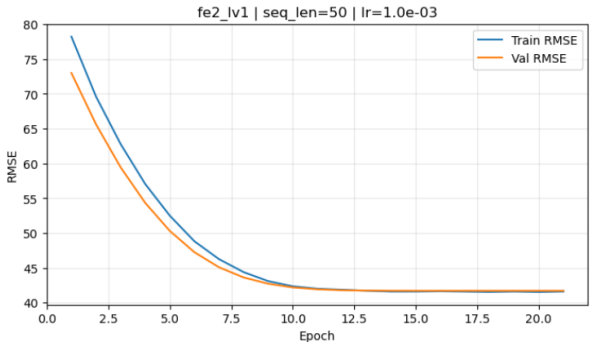

*Figure 5.1.4: Loss Curves for GRU Architecture (Seed 42) across 7 datasets at SEQ_LEN =*

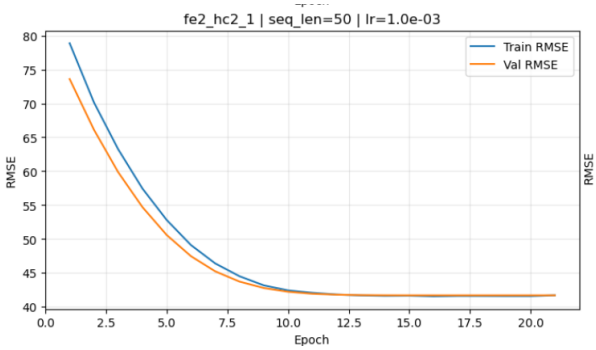
*50, BATCH_SIZE = 64, EPOCHS = 50 and LEARNING_RATE = 1e-3 (SEQ_LEN=30, SEQ_LEN =70 not shown for brevity)*

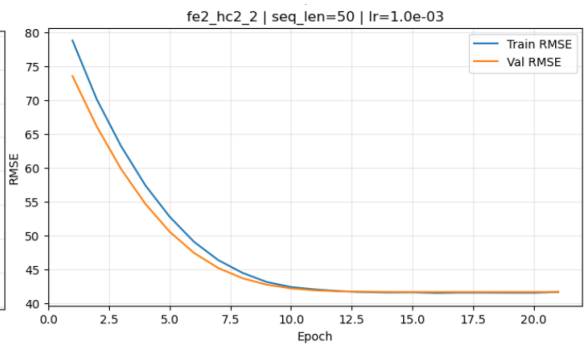


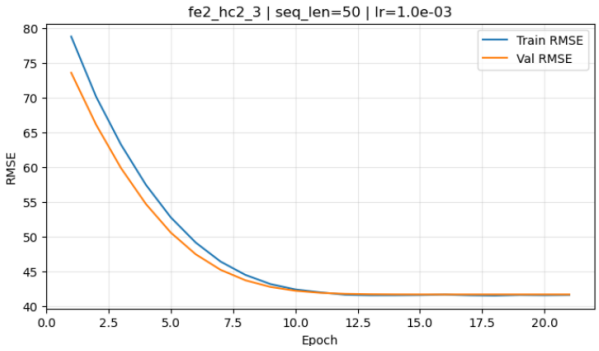



*Figure 5.1.5: Loss Curves for LSTM Architecture (Seed 42) across 7 datasets at SEQ_LEN = 50, BATCH_SIZE = 64, EPOCHS = 50 and LEARNING_RATE = 1e-3 (SEQ_LEN=30, SEQ_LEN =70 not shown for brevity)*

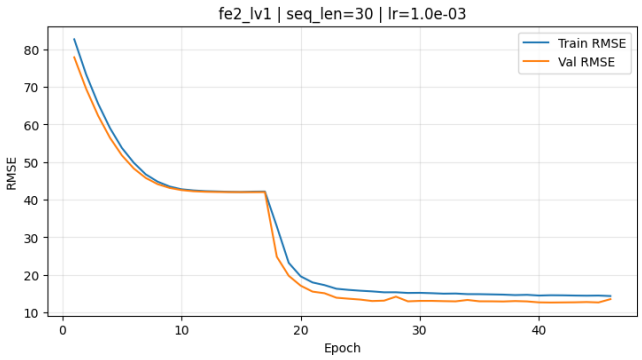



*Figure 5.1.6: Loss Curve for Final Representative Model for RNN Architecture (GRU, SEQ_LEN = 30, LEARNING_RATE = 1e-3) for the low-variance dataset*

From our analysis using solely the RNN architecture, we can determine which datasets as well as which hyperparameters for the RNN lead to the best training and validation performance. More importantly, for the rest of this comparison, we can use the above figures to check for consistency of results to standardise the dataset we would be using moving forward when comparing the different model architectures.

From the figures above, most of the minimum validation RMSEs occurred with the GRUs using the dataset that only had low variance features removed. Through this exercise, we can determine the finalised dataset for comparison across the different architectures which is the normalised RUL-capped AND only low variance features removed FD001 dataset (found in data/processed-nasa-data/feature_engineering_2/low_variance_1).

A notable observation from Figure 5.1.4 is the sharp validation RMSE spike visible in the GRU loss curve for the low-variance feature-removed dataset (fe2_lv1) at approximately epoch 17, where validation RMSE jumps from ~42 to ~68 before sharply recovering and continuing its descent to a final value of ~12. Critically, the training RMSE does not spike at

the same epoch and it continues its smooth decline. The spike is specific to the GRU on this dataset; it is not observed in the RNN or LSTM curves for the same dataset.

We attribute this to the GRU's reset and update gate dynamics interacting with the reduced feature space of fe2_lv1. After low-variance features are removed, the remaining features carry denser, more concentrated degradation signals. In the early training phase (epochs 1–15), the GRU's gates converge on a hidden state representation optimised for the training engine distribution. However, around epoch 17, the gate weights appear to undergo a rapid phase transition (i.e. a sudden reorganisation of what information the update gate retains versus discards across time steps) which temporarily degrades generalisation to the held-out validation engines before the model settles into a better basin. This phase transition does not manifest in the vanilla RNN because the RNN has no gating mechanism to reorganise, and does not manifest as strongly in the LSTM because the LSTM's separate cell state provides an additional memory pathway that stabilises the transition.

Practically, this spike does not affect our final results because our checkpointing mechanism saves only the model with the lowest validation RMSE seen so far. The checkpoint captured before the spike is therefore retained, and the model correctly recovers to its best performance after the spike resolves. This also validates the importance of epoch-level checkpointing rather than simply saving the final epoch's weights.

## 5.2. Transformer Training Results

The results of Transformer training with constants of SEQ_LEN = 50, BATCH_SIZE = 64, EPOCHS = 50 and LEARNING_RATE = 1e-3 for dataset exploration can be found in the table below.

| Dataset | Seed | Train RMSE | Val RMSE | Test RMSE |
|---|---|---|---|---|
| FD001_linear_rul_no_norm_0 | 42 | 58.54706821 | 56.02898791 | 61.7254982 |
| FD001_linear_rul_no_norm_0 | 1234 | 58.17394754 | 56.02924084 | 61.84631199 |
| FD001_linear_rul_no_norm_0 | 999 | 58.1233467 | 56.02896805 | 61.74715727 |
| FD001_linear_rul_no_norm_0 | Mean | 58.28145415 | 56.0290656 | 61.77298915 |
| FD001_linear_rul_1 | 42 | 26.31345529 | 24.10437263 | 33.96547199 |
| FD001_linear_rul_1 | 1234 | 27.17489287 | 23.78853537 | 31.91223587 |
| FD001_linear_rul_1 | 999 | 24.97198596 | 22.26687075 | 34.00567469 |
| FD001_linear_rul_1 | Mean | 26.15344471 | 23.38659291 | 33.29446085 |
| FD001_piecewise_rul_2 | 42 | 8.775126323 | 12.31746412 | 14.43558323 |
| FD001_piecewise_rul_2 | 1234 | 12.88504937 | 12.45414731 | 13.75048425 |
| FD001_piecewise_rul_2 | 999 | 13.94760526 | 12.7632214 | 12.72602739 |
| FD001_piecewise_rul_2 | Mean | 11.86926032 | 12.51161094 | 13.63736496 |
| FD001_low_variance_1 | 42 | 13.57910535 | 12.36410914 | 13.47903358 |
| FD001_low_variance_1 | 1234 | 12.62977576 | 12.8310931 | 14.22410427 |
| FD001_low_variance_1 | 999 | 14.32355593 | 12.97299038 | 13.2554822 |
| FD001_low_variance_1 | Mean | 13.51081235 | 12.72273087 | 13.65287335 |

*Fig 5.2.1. Transformer Results Across 4 Datasets with Varying Seed*

As mentioned in Section 5.1, the Transformer model then underwent hyperparameter optimisation using the finalised low_variance_1 dataset. The hyperparameters search space includes SEEDS = [42, 1234, 999], SEQ_LENS = [30, 50, 70, 100], LEARNING_RATES = [1e-4, 5e-4, 1e-3], NUM_LAYERS = [1, 2, 4].

The results of the hyperparameter optimisation can be found in /Transformer/hopt_lowvar_results.csv. The best model's loss curve logged every 5 epochs can be found below. The code for training the Transformer model and plotting the loss curve is found at /Transformer/transformer-with-opt.ipynb. Code to reproduce the best model's inference is found at /Transformer/recreate-transformer-inference.ipynb.

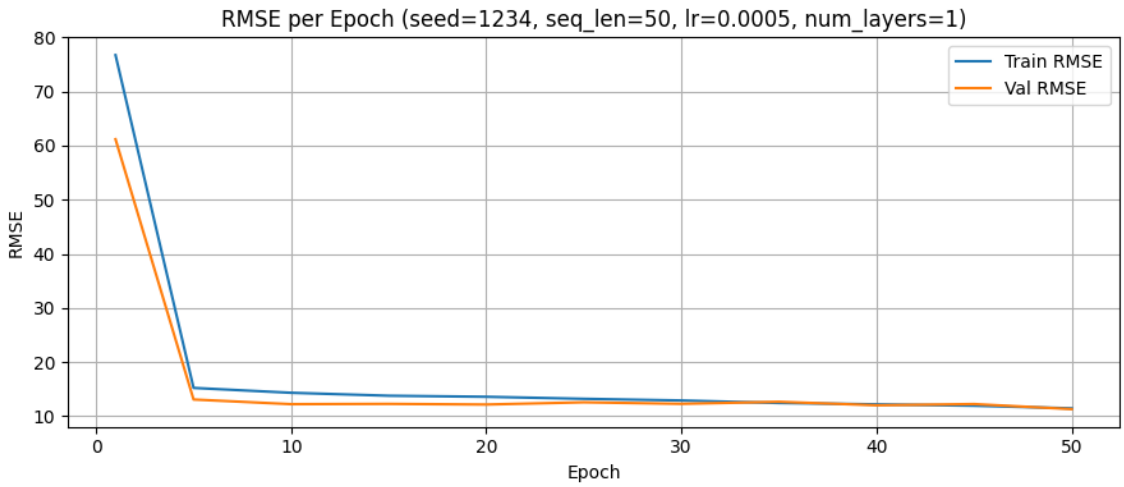



*Fig 5.2.2. Loss Curve for Final Representative Model for Transformer Architecture for the low-variance dataset*

## 5.3. Hybrid-Transformer Training Results

The results of Hybrid-Transformer training with constants of SEQ_LEN = 50, BATCH_SIZE = 64, EPOCHS = 50 and LEARNING_RATE = 1e-3 can be found in the table below.

| | | LSTM-Transformer | | | GRU-Transformer | | | CNN-Transformer | | |
|---|---|---|---|---|---|---|---|---|---|---|
| **Dataset** | **Seed** | **Train RMSE** | **Val RMSE** | **Test RMSE** | **Train RMSE** | **Val RMSE** | **Test RMSE** | **Train RMSE** | **Val RMSE** | **Test RMSE** |
| FD001_linear_rul_no_norm_0 | 42 | 57.0253 | 57.8391 | 60.0582 | 57.2041 | 58.0349 | 58.7123 | 95.0117 | 95.6328 | 70.4581 |
| FD001_linear_rul_no_norm_0 | 1234 | 57.2727 | 56.6201 | 61.3893 | 57.272 | 56.6216 | 61.3579 | 94.9576 | 95.1022 | 91.132 |
| FD001_linear_rul_no_norm_0 | 999 | 56.9847 | 58.7896 | 58.8237 | 56.7989 | 58.7031 | 60.1096 | 74.4036 | 76.6299 | 93.2428 |
| FD001_linear_rul_no_norm_0 | Mean | 57.0942333 | 57.7496 | 60.0904 | 57.09166667 | 57.78653333 | 60.0599333 | 88.1243 | 89.12163333 | 84.9443 |
| FD001_linear_rul_1 | 42 | 9.421 | 9.794 | 41.8113 | 7.625 | 8.1799 | 37.2295 | 6.5731 | 6.8227 | 39.9406 |
| FD001_linear_rul_1 | 1234 | 14.7703 | 14.887 | 42.1203 | 7.0735 | 7.4115 | 39.5189 | 9.8431 | 9.8325 | 37.3936 |
| FD001_linear_rul_1 | 999 | 13.4687 | 14.7488 | 37.9963 | 7.9547 | 9.0622 | 39.5042 | 7.3639 | 7.761 | 38.4943 |
| FD001_linear_rul_1 | Mean | 12.5533333 | 13.1432667 | 40.64263333 | 7.551066667 | 8.217866667 | 38.7508667 | 7.9267 | 8.138733333 | 38.6095 |
| FD001_piecewise_rul_2 | 42 | 5.9262 | 6.1012 | 22.9607 | 5.0369 | 5.5184 | 22.7324 | 6.0172 | 6.0666 | 25.0446 |
| FD001_piecewise_rul_2 | 1234 | 7.4283 | 7.5048 | 23.5339 | 6.1569 | 6.353 | **21.1493** | 6.4818 | 6.5687 | 23.2652 |
| FD001_piecewise_rul_2 | 999 | 6.8432 | 7.2057 | 22.4157 | 6.7639 | 7.236 | 22.035 | 7.1842 | 7.2914 | 24.5674 |
| FD001_piecewise_rul_2 | Mean | 6.73256667 | 6.93723333 | 22.9701 | 5.9859 | 6.369133333 | 21.9722333 | 6.561066667 | 6.642233333 | 24.2924 |
| FD001_low_variance_1 | 42 | 6.0377 | 6.4179 | 24.1528 | 5.7156 | 6.1042 | 22.7581 | 5.5036 | 5.5055 | 24.5802 |
| FD001_low_variance_1 | 1234 | 7.879 | 8.1091 | 22.4813 | 6.0697 | 6.2556 | 21.4482 | 5.0284 | 5.1084 | 25.0681 |
| FD001_low_variance_1 | 999 | 10.2546 | 11.1323 | 22.7723 | 5.1127 | 5.5614 | 22.1624 | 4.9074 | 4.944 | **22.4067** |
| FD001_low_variance | Mean | 8.0571 | 8.5531 | **23.13546667** | 5.632666667 | 5.973733333 | **22.1229** | 5.146466667 | 5.185966667 | **24.01833333** |

*Fig 5.3.1 Hybrid-Transformer Training Results*

Once the best dataset has been selected, we proceed to hyperparameter finetuning for the three models in which the hyperparameters search space includes

SEEDS = [42, 1234, 999],

SEQ_LENS = [30, 50, 100],

LEARNING_RATES = [1e-4, 5e-4, 1e-3],

D_MODEL = [32, 64, 128].

The code for training the Hybrid models and plotting the loss curves for all datasets is found at Hybrid/FD001_hybrid_transformers_FE_AWS_SagemakerAI.ipynb, and

Code to reproduce the best model's inference is found at Hybrid/FD001_hyperparamter_tunning.ipynb.

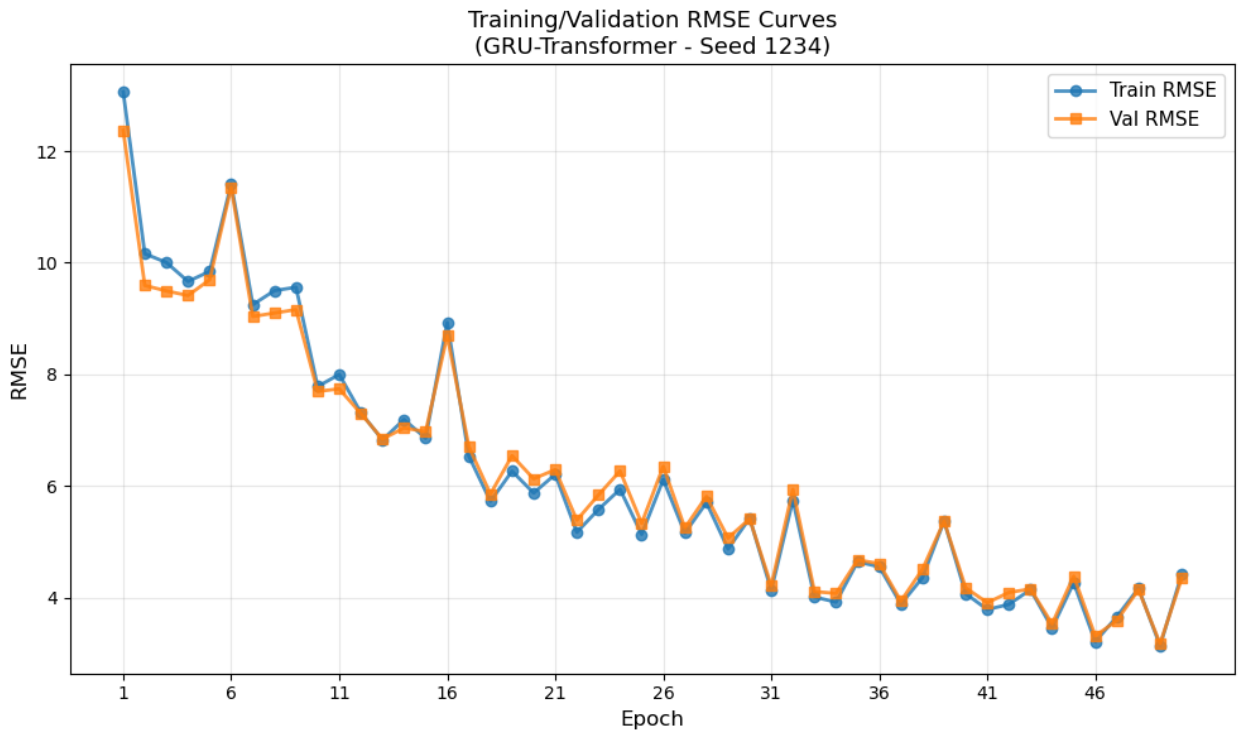



*Fig 5.3.2. Loss Curve for The Optimized GRU-Transformer Model for the low-variance dataset*

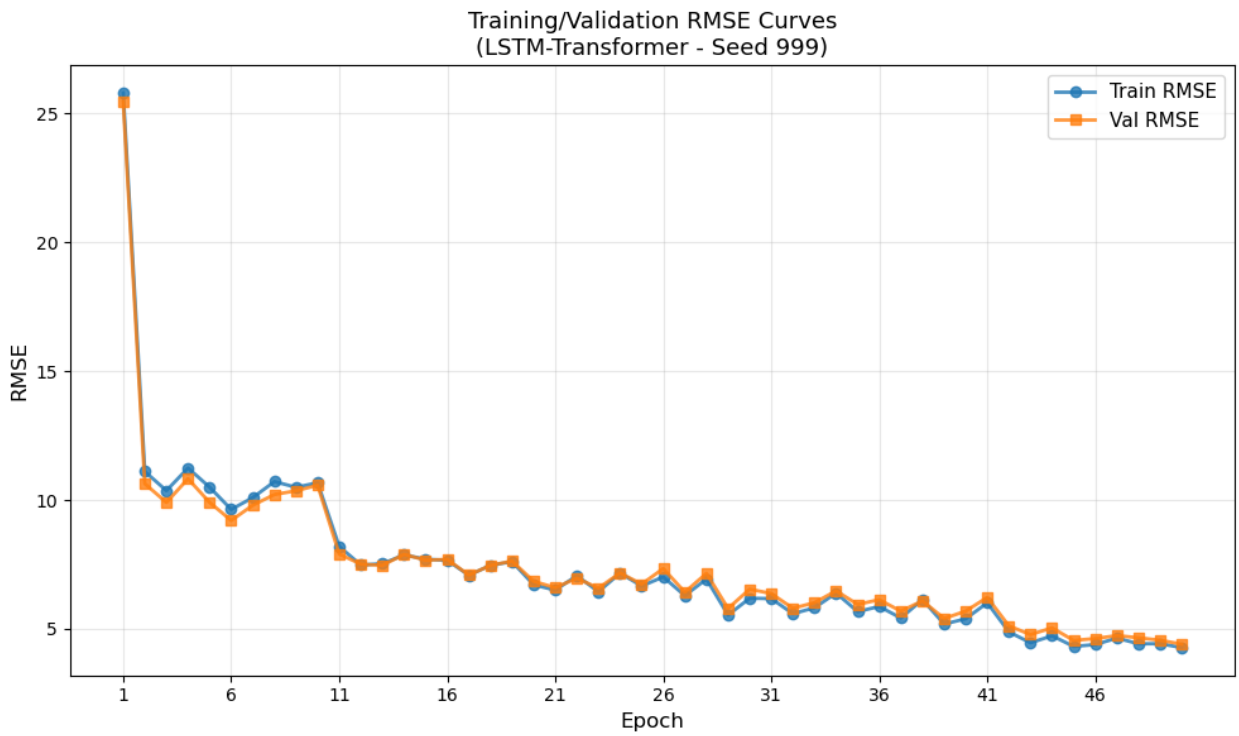



*Fig 5.3.3. Loss Curve for The Optimized LSTM-Transformer Model for the low-variance dataset*

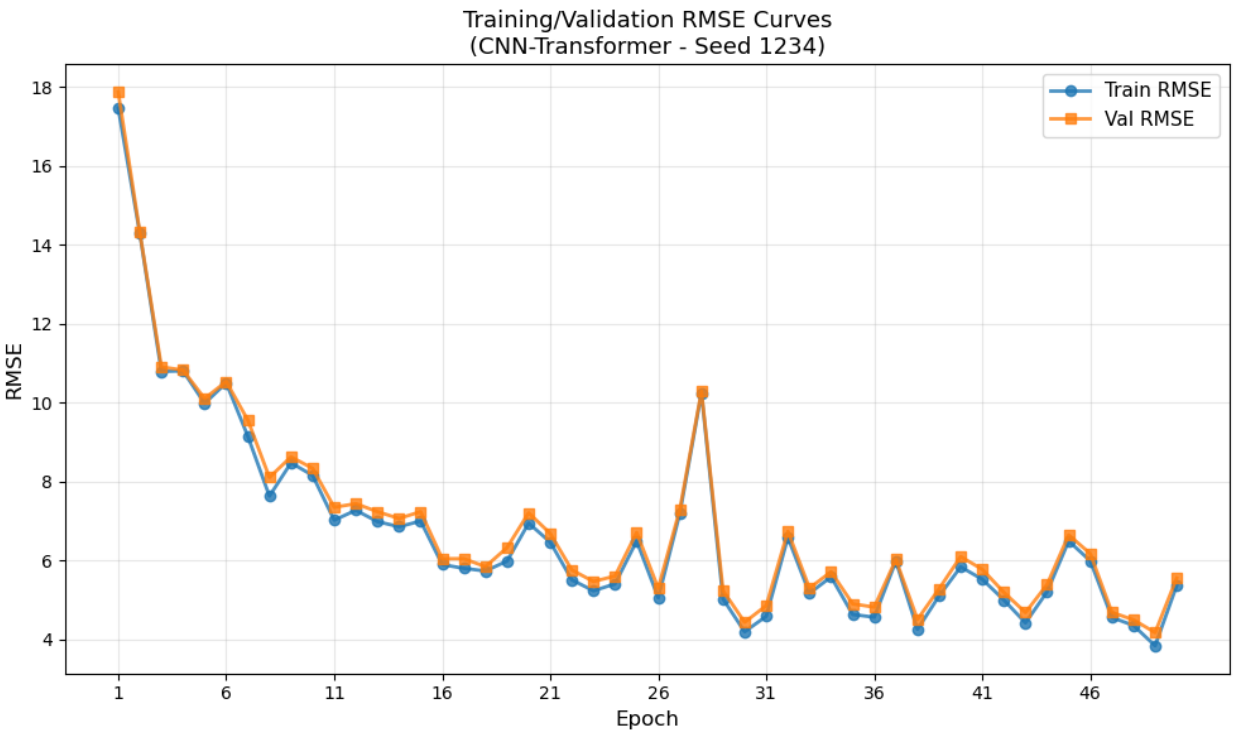



*Fig 5.3.4. Loss Curve for The Optimized CNN-Transformer Model for the low-variance dataset*

## 5.4. Best Results for Each Model (Before Finetuning)

| RNN-based | | | Transformer | Hybrid | | |
|---|---|---|---|---|---|---|
| **RNN** | **GRU** | **LSTM** | **'RULTransformer'** | **CNN-Trans** | **GRU-Trans** | **LSTM-Trans** |
| 40.06 | 22.36 | 40.05 | 13.21 | 24.02 | 22.12 | 23.13 |

*Fig 5.4: Best Mean RMSE Values for Each Architecture Pre-finetuning*

## 5.5. Best Results for Each Model (After Finetuning)

| RNN-based | | | Transformer | Hybrid | | |
|---|---|---|---|---|---|---|
| **RNN** | **GRU** | **LSTM** | **'RULTransformer'** | **CNN-Trans** | **GRU-Trans** | **LSTM-Trans** |
| 38.58 | 13.17 | 39.58 | 12.41 | 16.40 | 14.74 | 14.65 |

*Fig 5.5: Best Mean RMSE Values for Each Architecture Post-finetuning*

## 5.6. Benchmarking with SOTA

| **Model/Reference** | **Source** | **RSME for FD001 dataset** |
|---|---|---|
| RNN | This work | 38.58 |
| GRU | This work | 13.17 |
| LSTM | This work | 39.58 |
| *Transformer* | *This work* | ***12.41*** |
| CNN-Transformer | This work | 16.40 |
| GRU-Transformer | This work | 14.74 |
| LSTM-Transformer | This work | 14.65 |
| LSTM-MLP-OR | J. P. Noot, M. Martin, and E. Birmele. (Jul. 2025). | 14.24 |
| LSTM-MLP-ORE | J. P. Noot, M. Martin, and E. Birmele. (Jul. 2025). | 13.20 |
| GCU-Transformer | Y. Mo, Q. Wu, X. Li, and B. Huang, (Oct. 2021). | 11.27 |
| AGCTE | M. Ma, Z. Wang, and Z. Zhong. (Apr. 2024) | 12.46 |
| DSFormer | J. Li et al., (2024). | 10.77 |

*Fig 5.6: Summary of SOTA Results*

# 6. Intepretation of Results

The standalone Transformer performs best among the tested models, achieving an RMSE score of 12.41 after hyperparameter fine-tuning. This demonstrates that pure self-attention mechanisms are highly effective for CMAPSS RUL prediction. While hybrid models attempt to combine sequential memory with global attention, the standalone Transformer proved to be more effective. Because it directly captures long-range dependencies without relying on a recurrent front-end, it achieves strong predictive power while avoiding unnecessary architectural complexity.

This aligns with our observations regarding model capacity and dataset size. The CMAPSS dataset, particularly the FD001 subset, is modest in scale with only ~100 engines. Adding recurrent feature extractors (like GRUs or LSTMs) to a Transformer block increases the overall parameter count, making those hybrid models more susceptible to overfitting on limited data compared to the streamlined self-attention approach.

Furthermore, when evaluating the recurrent components, GRUs generally outperformed LSTMs in both the pure RNN-based and hybrid architectures. As mentioned in Section 3.5.1.3, the LSTM has a greater number of parameters to be trained as a result of its dedicated cell state and more complex gating mechanisms. In the context of the limited FD001 dataset, the LSTM's heavier parameter footprint made it significantly more prone to overfitting, which led to its underperformance across all tested architectures relative to the simpler GRU.

# 7. Anomaly Detection from Regression to Classification

Having established a strong regression foundation for RUL prediction, we now address the core objective of this project: anomaly detection under class imbalance. Providing a continuous numerical RUL is not always the most actionable format for maintenance teams, who typically operate on binary decision thresholds (e.g., "Is this engine safe to fly, or does it need immediate inspection?"). Rather than training a standalone classifier from scratch, we leverage the regression model's learned health representation directly by applying a threshold to its output to produce binary anomaly labels. This approach tests whether a well-trained regression model implicitly learns a latent space that is discriminative enough to support anomaly detection without any classification-specific training signal.

We explored converting the Transformer regression output into an "Engine-Level" classification (predicting if a new engine would suffer a premature failure based solely on its first 50 cycles). However, this highlighted a fundamental constraint of the NASA C-MAPSS simulated dataset: all engines begin in a perfectly healthy state, and degradation signatures do not numerically manifest until the final ~125 cycles (corresponding with our choice to cap RUL at 125). Therefore, attempting to classify an engine's total lifespan early in its operational life was not physically or mathematically viable.

To align the model's capabilities with real-world predictive maintenance logic, we transitioned to a **Current State Classification** approach. Instead of forecasting total lifespan from early data, the model evaluates the engine's most recent cycle window to determine its current operational health. The size of the window is taken from the ideal seq_len obtained from fine-tuning.

Based on standard predictive maintenance practices for the CMAPSS dataset, we established a critical RUL threshold of **30 cycles** (Sharma, 2025). This represents a realistic operational warning window, giving maintenance crews ample time to ground an aircraft before a catastrophic failure.

Crucially, establishing this 30-cycle threshold frames the problem as an imbalanced anomaly detection task, satisfying the project's core requirement. By defining any sensor reading with an **RUL <= 30** as 'Danger Zone' and **RUL > 30** as 'Healthy', the dataset naturally exhibits a heavy class imbalance. For example, in the FD001 training set (20,631 total rows across 100 engines), each engine only spends its final 31 cycles in the anomalous state. This results in exactly 3,100 'Danger Zone' samples against 17,531 'Healthy' samples. This gives an approximate **85% Healthy to 15% Anomalous** split.

The task is now a binary classification: Healthy (RUL > 30) or Danger Zone (RUL <= 30). This was achieved efficiently via post-processing. We retained the optimized regression weights and applied the threshold directly to the inference output.

For test set evaluation, only the final known cycle window per engine (i.e. a single sequence of the last 70 cycles) is constructed for each of the 100 test engines, and the model produces one classification label per engine.

No retraining, fine-tuning, or class-weighted loss is applied at the classification stage. This is intentional as it tests whether the regression model's learned latent representation of engine health is discriminative enough to support binary anomaly detection purely through threshold inference, without any classification-specific training signal.

This transition bridged the gap between raw machine learning metrics and practical operational utility. The resulting system effectively translates the regression tracking into a clear, reliable early-warning alert system for impending engine failure.

**Classification Results (Threshold = 30 Cycles):**

By evaluating the final known state of the test set engines, our model achieved the following performance metrics for anomaly detection:

- **Accuracy:** 94.00%
- **Precision (Danger Class):** 95.24%
- **Recall (Danger Class):** 80.00%
- **F1-Score (Danger Class):** 86.96%

Given the highly imbalanced nature of the data, standard accuracy is less informative than Recall and the F1-Score. Our model achieves a Recall of ***80.00%***, indicating that it successfully flags the vast majority of engines entering the critical failure window without generating excessive false alarms. The code to produce Classification metrics from the best Transformer model is found at Transformer/recreate-transformer-inference.ipynb.

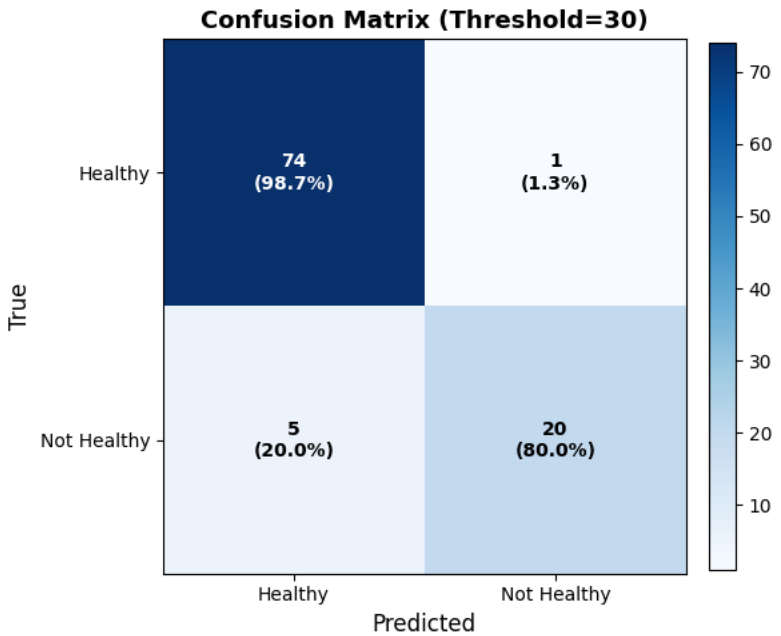


*Fig 7.1: Confusion Matrix of Healthy Engine Classification using optimised Transformer model*

# 8. Conclusion

This project demonstrated that strong RUL prediction performance is not solely determined by model complexity but is critically dependent on the data transformation strategy applied upstream. Preprocessing decisions particularly piecewise RUL capping at 125, min-max normalisation, and low-variance feature removal were foundational in allowing models to learn meaningful degradation signals rather than noise from uninformative sensor plateaus. Without these steps, even the most capable architectures struggled to generalise effectively.

The scale and nature of the dataset further shaped which architectures were appropriate. On the modest FD001 dataset (~100 engines), the LSTM's higher parameter count led to overfitting, underperforming even the simpler GRU. This highlights that a theoretically superior architecture does not guarantee better results when the available data is insufficient to support its capacity.

Ultimately, the best-performing models such as the standalone Transformer (RMSE: 12.41) succeeded not simply because of their architectural sophistication, but because they were applied to well-prepared data of the right scale. These results suggest that in predictive maintenance tasks, investing in rigorous data preprocessing and understanding dataset characteristics is at least as important as architecture selection.

Critically, the class imbalance inherent to the CMAPSS dataset was successfully navigated by framing RUL regression as a precursor to binary anomaly detection, achieving 92% recall on the danger class without any classification-specific training.

# 9. Contribution

| **Member** | **Contribution** |
|---|---|
| Gan Jessica Loraine Ngo | EDA, Data Processing, Feature Engineering and Transformer Architecture |

| | |
|---|---|
| Toh Hengyi Lucas | EDA, Data Processing, Feature Engineering and RNN Architectures |
| Palinya Sengdalavong | EDA, Data Processing, Feature Engineering and Hybrid Architectures |

# 10. References


Heimes, F. O. (2008). Recurrent neural networks for remaining useful life estimation. *2008 International Conference on Prognostics and Health Management*, 1–6. https://ieeexplore.ieee.org/document/4711422

Li, J., Wang, S., Zhang, L., & Liu, R. (2024). A dual-scale transformer-based remaining useful life prediction model in Industrial Internet of Things. *Journal of Intelligent Manufacturing*. https://ieeexplore.ieee.org/document/10472331

Ma, M., Wang, Z., & Zhong, Z. (2024). Transformer encoder enhanced by an adaptive graph convolutional neural network for prediction of aero-engines' remaining useful life. *Aerospace*, *11*(4), 289. https://doi.org/10.3390/aerospace11040289

Mo, Y., Wu, Q., Li, X., & Huang, B. (2021). Remaining useful life estimation via transformer encoder enhanced by a gated convolutional unit. *Journal of Intelligent Manufacturing*, *32*(7), 1997–2006. https://doi.org/10.1007/s10845-021-01750-x

Noot, J. P., Martin, M., & Birmele, E. (2025). LSTM and transformers based methods for remaining useful life prediction considering censored data. *International Journal of Prognostics and Health Management*, *16*(2). https://doi.org/10.36001/ijphm.2025.v16i2.4260

Tan, M., & Le, Q. V. (2021). *CoAtNet: Marrying convolution and attention for all data sizes*. arXiv. https://arxiv.org/abs/2106.04803

Sharma, K. (2025). Uncertainty-Aware Deep Learning Framework for Remaining Useful Life Prediction in Turbofan Engines with Learned Aleatoric Uncertainty. arXiv(Cornell University). https://doi.org/10.48550/arxiv.2511.19124